



贵州理工学院

Guizhou Institute of Technology

贵州理工学院 本科毕业论文（设计）

（2025 届）

设计题目：基于 RK3588NPU 的人员离岗行为检测系统设计
与实现

学 院： 大数据学院

专 业： 智能科学与技术

班 级： 智能 211

学 号： 2021240245

学生姓名： 熊海林

第一指导教师： 张良

第二指导教师： 吴燕杰

2025 年 5 月 25

贵州理工学院本科毕业论文（设计） 诚信责任书

本人在此郑重声明：本人所提交的毕业论文（设计），乃是在导师给予指导的情况下，凭借自身之力完成相关研究后所成。毕业论文（设计）里但凡引用了他人不管是已经发表的成果、数据、观点，还是未发表的成果、数据、观点等内容，都已经清清楚楚地注明了其出处所在。

特此声明。

论文（设计）作者签名：熊海林

日期：2025 年 5 月 25 日

本科毕业论文（设计）AI 工具使用说明书

在本次写作过程中，为了提高内容生成的效率和质量，在确保符合学术规范的前提下，本人使用了 AI 工具，本人对 AI 工具生成内容负全部责任并确保论文原创性和学术诚信。以下是关于 AI 工具使用的详细说明：

一、工具名称：百度学术 AI、DeepSeek

二、使用范围及过程

1. 使用范围

引言框架结构搭建（百度学术 AI）

论文语言润色（DeepSeek）

2. 使用规范

所有引用文献均经过人工核对原始文献。

研究数据来源于网络公开数据集，未使用 AI 生成或修改原始数据。

关键研究结论均来自本人独立分析。

学生签名：

熊海林

日期：2025 年 5 月 25 日

目录

摘要	III
Abstract	IV
1 引言	1
1.1 研究背景及意义	1
1.2 研究的国内外现状和发展趋势	1
1.3 论文研究内容与结构安排	2
2 系统分析	4
2.1 主要技术分析	4
2.2 硬件分析	5
2.3 软件分析	8
2.4 算法分析	13
2.4.1 目标检测算法	13
2.4.2 跟踪算法	16
3 系统设计与实现	19
3.1 人员检测模型及离岗检测算法设计	19
3.1.1 人员检测模型	20
3.1.2 跟踪算法及离岗检测	23
3.2 检测模型转换	25
3.2.1 PyTorch 模型转换为 ONNX 模型	25
3.2.2 ONNX 模型转换为 RKNN 模型	27
3.3 检测模型的开发板移植	29
3.3.1 开发板环境配置	29
3.3.2 开发板环境与模型移植效果测试	30
4 系统测试	31
4.1 本地人员检测模型以及离岗检测算法测试	31
4.1.1 图片检测	31
4.1.2 视频检测	33

4.1.3 实时检测	34
4.2 开发板测试	34
5 结论与展望	37
5.1 总结	37
5.2 展望	37
参考文献	39
致谢	40

基于 RK3588NPU 的人员离岗行为检测系统设计与实现

摘要

在工业生产以及机械操作的整个过程当中，常常出现负责特定工程器械的操作人员私自脱离自己工作岗位的现象，这种情况给生产安全以及其他的施工人员的人身安全都带来了隐患。虽然已经运用打卡离岗的监管机制，可在实际去执行时还是很难满足人员离岗行为监管的需求。因此设计一个能够对人员离岗情况进行检测的系统对于切实保障生产安全有着极为重要的意义。

本文设计和实现了人员离岗行为检测系统并将算法以及模型成功移植到 RK388NPU 硬件模块中。在人员离岗检测的系统框架中，通过引入 YOLO 算法实现对人员的识别以及利用 DeepSORT 算法增强系统对于多目标的跟踪能力，使得系统整体的检测性能得到进一步的加强，提升系统在人员检测以及人员跟踪方面的效果。在模型移植的阶段，将检测模型转换为 RK3588NPU 所支持的神经网络结构，充分利用 RK3588NPU 的计算性能。通过对检测系统进行相应的测试，验证该系统的可行性与实用性，也给后续相关的应用提供了一些参考的依据。

关键词：RK3588NPU；人员离岗行为检测；YOLO 算法；DeepSORT 算法；

Design and Implementation of Personnel Absence Behavior Detection System Based on RK3588NPU

Abstract

During the entire process of industrial production and mechanical operation, operators in charge of specific engineering equipment often leave their work posts without authorization, which poses potential safety hazards to production safety and the personal safety of other construction personnel. Although a time-card based departure supervision mechanism has been adopted, it still fails to meet the actual needs of monitoring personnel absence behaviors in practical implementation. Therefore, designing a system for detecting personnel absence is of great significance to effectively ensure production safety.

This article designs and implements a personnel absence behavior detection system, and successfully deploys the algorithms and models used in the system to the RK3588NPU hardware module. In the system framework for personnel absence detection, the YOLO algorithm is introduced to realize personnel recognition, and the DeepSORT algorithm is utilized to enhance the system's multi-target tracking capability, thereby further improving the overall detection performance and the effectiveness of personnel detection and tracking. During the model transplantation phase, after converting the detection model into a neural network structure supported by the RK3588NPU, the computing performance of the RK3588NPU is fully leveraged. Through corresponding tests on the detection system, its feasibility and practicality are verified, providing references for subsequent related research.

Keywords: RK3588NPU; Detection of Personnel Off-duty Behavior; YOLO Algorithm; DeepSORT Algorithm;

1 引言

1.1 研究背景及意义

在社会不断发展的历程中,人员私自离岗行为在给相关行业造成巨大的经济损失的同时,也给部分行业内的工作人员埋下了人身安全隐患^[1],对于人员离岗行为的监管在工业领域的安全生产方面、公共安全的管理层面以及服务行业的运营环节都有着极为关键的作用。就制造业领域而言,操作人员私自离岗而导致的操作中断情况往往会带来巨大的经济损失,在此过程中通常还伴随着极大的安全风险。对服务行业来讲,人员私自离岗导致的服务质量下降会相应的降低客户的产品体验,品牌形象将会受到极为严重的损害。除此之外,在化工生产这类高危性质的行业之中,擅自离岗行为有可能会引发灾难性后果^[2]。为了降低人员离岗带来的危害,在生产过程中设置对应的监管制度来检测离岗行为的发生成为了必然。

传统的人员离岗监测办法存在不少缺陷。在依靠人工来开展离岗监测工作的时候,往往会出现监测不到的区域,就打卡系统而言,它只能记录固定的时间节点,对于上班时间段内人员的离岗行为无法实施有效的监管举措。在引入视频监控设备之后,在一定程度上让监管的效果有所提升,可是依赖人工去逐一筛查视频数据的做法,依旧存在着一些弊端,而且还会在很大程度上受到监管人员的身体状态的影响。在持续进行监控的这个过程当中,很容易出现因为监管人员视觉疲劳进而使得漏检或者误判的情况,实际所能达到的监管效果就很难完全符合使用方面的各项需求。

随着深度学习技术的不断发展,将深度学习技术在人员离岗监测领域运用彰显出明显的优势。深度学习所涉及的计算机视觉技术能够自动对视频流加以解析,借助于多层神经网络可以对大量的相关数据展开学习^[3],模型在经过这一系列的学习操作之后可以十分有效地识别出离岗行为。在一定程度上对传统人工视频筛查存在的不足之处予以弥补,使得监管人员身体状态给监管效果带来的影响得以降低,进而促使监管系统整体的监测效果获得明显的提升^[4]。

1.2 研究的国内外现状和发展趋势

近些年来,深度学习技术飞速发展,与之相关的人员离岗监测技术也有很大的进展,这一技术在许多行业得到应用。针对制造行业来讲,不少工厂依靠该技术对生产

线上各个岗位的员工实施实时监测，一旦操作员擅自离岗，系统就能快速地识别出来并且发出警报，管理层便能及时了解情况进而采取相应的举措，极大的降低因为离岗而致使设备出现故障的风险，有力地保障工业产能以及工人安全^[5]。在服务行业当中离岗监测系统也受到大量使用，通过对柜员、医护人员等服务人员的在岗状态展开不间断监测，能及时监测离岗行为并加以提醒纠正，有效地防止因离岗而出现的服务空档期，有效提升了服务质量。

当下，国内相关技术的发展有着多技术融合、智能化不断深入以及行业定制化持续拓展等趋向。一方面，深度学习和物联网、大数据等技术在相互融合，由此构建出更为完善且智能的人员离岗监测网络。借助物联网技术，各类监测设备能够实现相互连通，把人员离岗的相关信息传送到云端的大数据平台上，经过深度学习算法分析处理后不但能够精准地判别离岗行为，而且还能够结合历史数据来挖掘离岗的规律，给企业进行人员管理优化给予强有力的依据。另一方面，伴随着算法持续地优化以及算力有了大幅的提升，人员离岗监测系统的智能化程度也在不停地提高，不但能够达成离岗检测的目的，而且还能够针对人员离岗前的行为异常状况发出预警^[6]，提前察觉到潜在的离岗风险，从而为企业采取预防举措争取到足够的时间。

在国外人员离岗监测相关技术的研究中，在算法层面，基于 YOLO 系列算法的目标检测技术被广泛应用于工业场景，结合关键点识别算法 OpenPose 实现了对离岗监测的精细化。在此基础上，通过应用多目标跟踪算法整合运动信息与外观特征显著提升了复杂场景下的跟踪效果^[7]。在技术融合层面主要倾向于物联网与边缘计算结合，通过部署智能传感器与边缘计算设备实现了对工厂、医院等场景的实时数据采集与本地分析，大幅减少云端传输延迟^[8]。

1.3 论文研究内容与结构安排

本次设计围绕人员离岗行为检测系统设计与实现展开。通过在开发板上移植检测模型实现对人员离岗行为检测，从而获取人员离岗行为信息，有效的降低相关工作人员的工作量。在面对突发情况时能够及时的发现并处理，减少损失、快速处理问题确保的正常运行，论文结构安排如下：

第一章引言，该章节在讲述研究背景与意义时，详细分析当前生产安全、公共安全及企业管理等领域中传统人员离岗检测的不足，以及引入视频监控后依然存在的缺陷，突出基于深度学习的人员离岗行为检测系统对提升管理效率和安全性的重要性。

在描述国内研究现状部分，讲述了人员离岗监测在国内外行业的应用情况以及主要改善方向，明确本次设计的内容与创新点。

第二章系统分析部分介绍涉及的相关技术。深度学习基础理论板块简要介绍其与机器学习的关系，在硬件分析部分分析 RK3588NPU 的多核异构处理器架构、NPU 单元的算力参数和低功耗特性以及相应开发板的具体硬件配置。在软件分析模块围绕神经网络介绍基本结构，逐步深入到卷积神经网络的原理及其在目标检测领域的应用优势。在算法分析部分，对于 YOLO 系列目标检测算法，解析其检测的高效架构以及如何通过特征网络实现多尺度目标检测，针对 DeepSort 跟踪算法阐述其匹配策略，以及如何结合表观特征提升跟踪准确性。

第三章系统设计与实现，该部分人员检测模型搭建以及离岗检测算法设计实现、人员模型转换以及检测模型的开发板移植三个部分展开。人员检测模型搭建及离岗检测算法设计环节介绍本次采用的神经网络模型并对模型进行修改，详细说明对数据集 YOLO 格式的处理操作以及利用预训练模型进行训练的过程。人员检测模型转换环节分步讲解 Pytorch 模型向 ONNX 模型转换时结构优化，简要讲解 ONNX 模型转换为 RKNN 模型过程中针对 RK3588NPU 硬件的量化处理。检测模型的开发板移植部分，描述开发板硬件的环境配置过程配置，对转换工具的安装过程进行说明。最后，通过编写图片检测代码完成对已经搭建环境和模型的初步测试。

第四章系统测试分为本地人员检测模型以及离岗检测算法测试和开发板测试两部分。本地人员检测模型以及离岗检测算法测试部分通过对模型针对不同类型文件的检测结果进行演示，以此来检验模型的适配度以及设计的离岗算法的可行性，为模型转换后的检测效果提供基础保证。开发板测试则模拟实际应用场景，检验系统在硬件资源受限情况下的稳定性、实时性和检测准确性，全面评估系统的有效性。

第五章总结本次设计的主要成果，该部分通过对系统设计完成部分的总结，讲述系统的主要设计流程，也明确 RK3588NPU 的人员离岗行为检测系统在适应性上的优势。同时，也客观分析系统在设计 and 实现过程出现的问题，最后对未来研究方向进行展望，提出引入更先进的目标检测算法、优化硬件与算法协同设计等具体优化策略，为后续研究提供参考。

2 系统分析

2.1 主要技术分析

机器学习属于一门涉及多领域的交叉学科，其中涵盖概率论、统计学、逼近论、凸分析以及算法复杂度理论等诸多学科领域。它着重对计算机模拟或者实现人类学习行为的方式展开研究，目的在于让计算机能够获取新的知识与技能并且对已有的知识结构加以重新组织，从而促使其自身性能不断得到提升。在人员离岗行为检测系统当中，机器学习技术发挥着十分重要的作用。

深度学习属于机器学习的分支领域，它主要是一种针对数据展开表征学习的方式，其与机器学习的关系如图 2.1 所示。借助搭建具备多层次结构的神经网络模型这一手段，深度学习模型能够从海量的数据当中去学习相对复杂的模式以及各类特征，其在人员离岗行为检测系统设计与实现的整个过程里，占据着十分重要的地位。

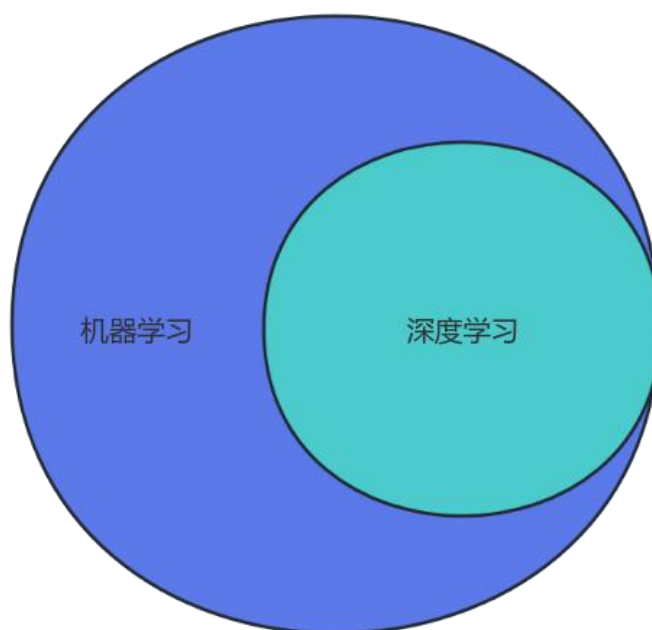


图 2.1 深度学习与机器学习的关系

嵌入式技术是一种把计算机硬件以及软件相互结合起来，随后嵌入到其他一些设备当中，进而实现特定功能的一种技术手段^[9]。RK3588NPU 属于一款颇为先进的嵌入式处理器，它为人员离岗行为检测系统打下坚实的硬件基础。其具备体积小巧、功

耗比较低、可靠性颇高的特点。在检测系统中，RK3588NPU 当中集成功能强大的神经网络处理单元，其能够以高效的方式去运行深度学习相关的模型。和传统的通用处理器对比来看，RK3588NPU 在针对深度学习算法方面进行了专门的优化处理，极大地提升模型的推理速度。

2.2 硬件分析

在硬件层面，RK3588NPU 充当着系统核心运行套件的角色。它依靠自身具备的 6Tops 算力以及对多精度的支持能够降低深度学习模型推理所耗费的时间。其拥有丰富的指令集并且采用低功耗设计，这都为模型的高效运行给予有力保障。在深度学习相关方面，RK3588NPU 对 PyTorch 框架是兼容的，这种设计使得开发者能够较为便捷地把已经训练好的模型进行移植。通过运用算子融合、模型量化等一系列优化手段，在运行 YOLO 系列目标检测模型的时候，就更容易实现高精度以及低延迟的实时处理成效，这也完全符合本次设计针对开发板的硬件方面的需求^[10] RK3588 核心板如图 2.2 所示。



图 2.2 RK3588 核心板

LubanCat-5-BTB 底板是 RK3588 核心板的承载依托之一，其设计主要围绕着硬件扩展性、数据处理能力以及多场景适配性等诸多方面来展开运作。从对硬件架构细致入微的布局规划，再到对丰富多样接口进行合理恰当的配置安排，直至卓越出色的

性能参数得以实现达成, 这些方面所呈现出的种种特性都为系统的设计与具体实现给予在硬件开发层面上的诸多便利, LubanCat-5-BTB 开发板如图 2.3 所示内容。该底板同 RK3588 芯片之间的深度适配有力的确保 RK3588 芯片能够将自身所具备的高算力优势充分发挥出来。

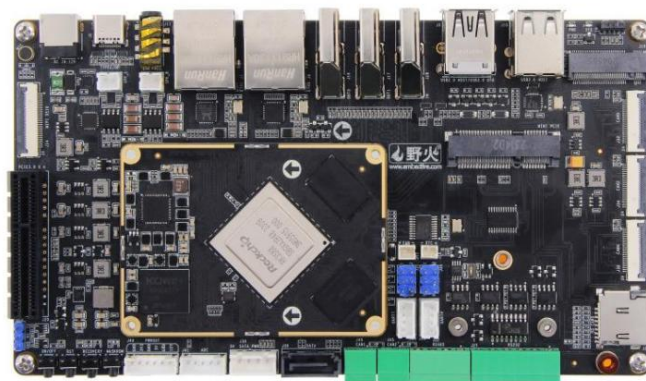


图 2.3 LubanCat-5-BTB 开发板

LubanCat-5-BTB 开发底板运用六层堆叠的设计方式, 从硬件层面给系统的稳定运行给予有力保障, 具体分层设计如图 2.4 所示。在其顶层位置主要放置主控芯片以及一些关键接口, 这种设计方便开发者能够较为迅速地和外设进行对接, 进而开展功能方面的验证工作; 第二层采用的是全平面接地设计, 这一设计构造形成电磁防护网, 切实有效地削减外部干扰给敏感信号所带来的影响。处于中间的第三层和第四层属于关键信号层, 这两层通过运用差异化线路布局的手段, 达成性能优化的目标。就低速控制信号而言, 始终维持着最短路径的状态, 最大程度地减少信号串扰情况的出现。第六层作为专门的电源层, 配备有多组相互独立的供电轨道, 再结合上经过特殊设计的电容阵列, 便能够为设备的运行提供较为稳定的供电条件。

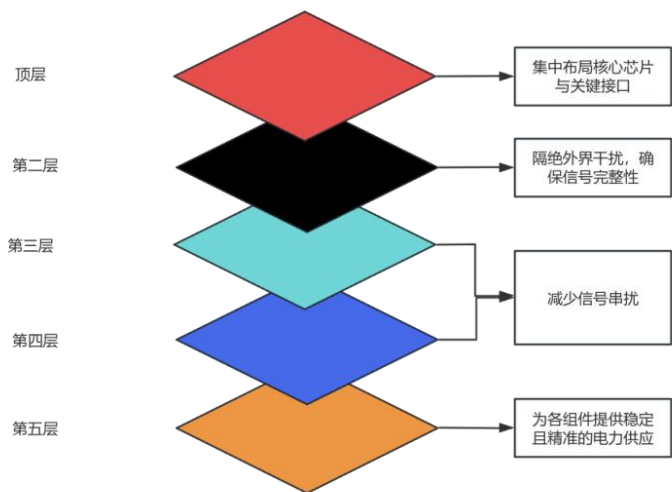


图 2.4 LubanCat-5-BTB 底板分层结构

LubanCat-5-BTB 开发板在接口设计方面彰显出颇高的实用性以及可拓展性。其配备的丰富接口配置，能够给开发者搭建多种多样的应用给予强有力的支撑，同时也为此次设计内容得以实现带来了诸多便利，图 2.5 所示为该开发板的部分接口情况。

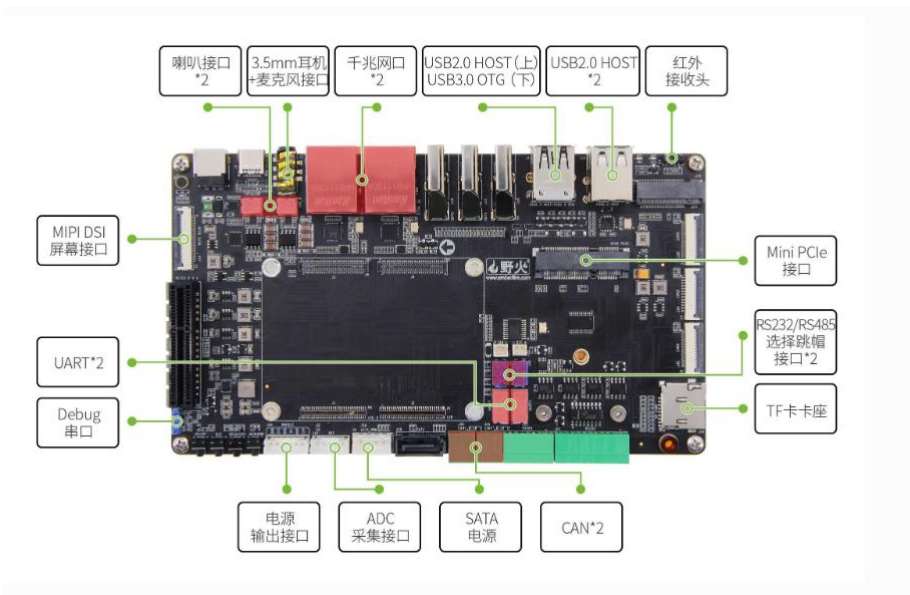


图 2.5 LubanCat-5-BTB 的部分接口

这款开发板所配备的通信模块，将千兆以太网接口进行集成处理，其能够支持自适应速率这一特性并且还具备远程唤醒功能。在工业自动化领域，当面临大量数据需要实时传输的要求时，它能够很好地予以满足，与此同时，借助网络管理手段还可以使设备能耗得以降低。在无线通信这个层面上，Wi-Fi 模块和蓝牙模块相互组合起来，这就为智能家居设备之间实现互联，为移动终端能够稳定且高效地接入给予相应的硬

件方面的有力支持。

在数据传输这个层面上，双模式的 USB 接口将通用性以及高性能充分地展现出来。其中，USB 2.0 接口能够灵活地接入像键盘、鼠标这类基础外设，以此来满足日常进行交互的需求。而 USB 3.0 接口则配备速率达到 5Gbps 的带宽，从而可以实现对高速存储设备进行快速的读写操作，在大数据分析类似的高负载场景当中发挥着极为重要的作用。此外，专用的 MIPI 接口通道还对高清显示输出和工业相机接入给予支持，该接口通道不但能够驱动多种规格的液晶屏以实现专业级别的图像显示，而且还可以连接高分辨率的摄像头模组，进而为系统的实现构建起可靠的数据采集通道。

扩展接口体系给予开发板颇为出色的硬件兼容能力。当完成对相应 GPIO 阵列的配置之后，开发板便能够连接诸如温湿度传感器、电磁继电器这类外设进而在环境监测以及智能控制等项目当中达成快速的硬件对接操作；PCIe 扩展槽对 SSD 硬盘、加密加速卡等高性能模块予以支持，于边缘计算的场景之中可以较为明显地提升数据存储的密度以及算法处理的效率。

2.3 软件分析

在软件层面，在实现人员离岗行为检测系统的时候我们运用神经网络。神经网络属于一种模拟神经元连接而形成的网状结构模型，它借助大量人工神经元彼此相连的方式，针对输入数据展开非线性变换方面的处理操作进而达成特定任务。这些神经元依靠权重、偏置等参数来完成连接设置，针对网络的输入施行特定的数据处理举措，好让最终所呈现出来的结果可以契合预定的目标要求。

神经网络大体上可分成输入层、隐藏层还有输出层这三个部分，如图 2.6 所示。输入层承担的任务是对待检测的数据予以接收。当接收到由输入层所提供的数据之后，隐藏层便会针对该数据开展特征提取以及变换方面的相关操作。而输出层则会依据隐藏层所提取出来的那些特征去生成与之相对应的预测结果。在这样一个完整的过程当中，处于同一层次的神经元彼此之间是不存在连接关系的，后一层的神经元会和前一层的全部神经元建立起连接，通过针对神经元连接通道上的权重加以设置的方式，来对整个神经网络最终的输出结果做出相应的调整。

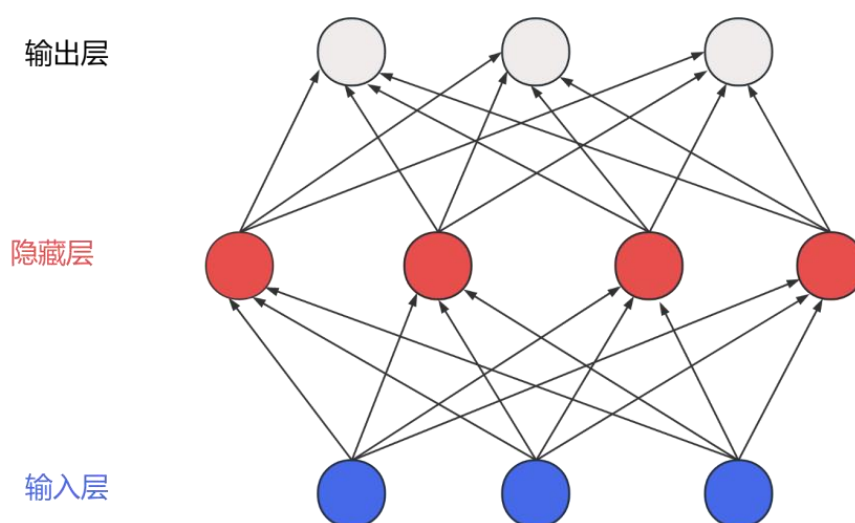


图 2.6 神经网络层次图

常用的神经网络有前馈神经网络、递归神经网络，还有卷积神经网络。前馈神经网络属于最为基础的神经网络类别，它的数据从输入层出发，经过隐藏层，而后传递到输出层，在这个传递的过程当中并不存在反馈结构。其网络架构相对来说比较简洁，网络中的数据呈现单向流动的态势，使得整体理解起来较为容易，比较适用于去处理基本的分类以及回归方面的任务。和前馈神经网络相比，递归神经网络在前馈神经网络的基础之上又引入反馈结构的，通过这种结构调整，数据就能够在网络当中循环着进行传递，递归神经网络也就具备了处理序列数据的能力。对于卷积神经网络而言，它是通过卷积层来提取输入数据的局部特征的，并且还借助池化层去降低特征的空间维度，以此来达成特征的层次化表示。卷积神经网络尤其适用于处理网格结构的数据。在图像识别、物体检测以及图像分割等计算机视觉相关的任务中，卷积神经网络所表现的效果格外显著^[11]。

卷积神经网络包含输入层、隐含层以及输出层^[12]。输入层主要承担接收数据的职责，输入的数据通常是阵列的形式。隐含层是由卷积层、激活层、池化层还有全连接层共同组成的，其负责针对输入的数据开展特征提取方面的工作。输出层会依据具体的任务类型来输出相应结果，比如在图像分类任务里会输出各个类别的概率分布情况，而在目标检测任务中则会输出目标的类别标签以及位置坐标^[13]可参照图 2.7 所示。

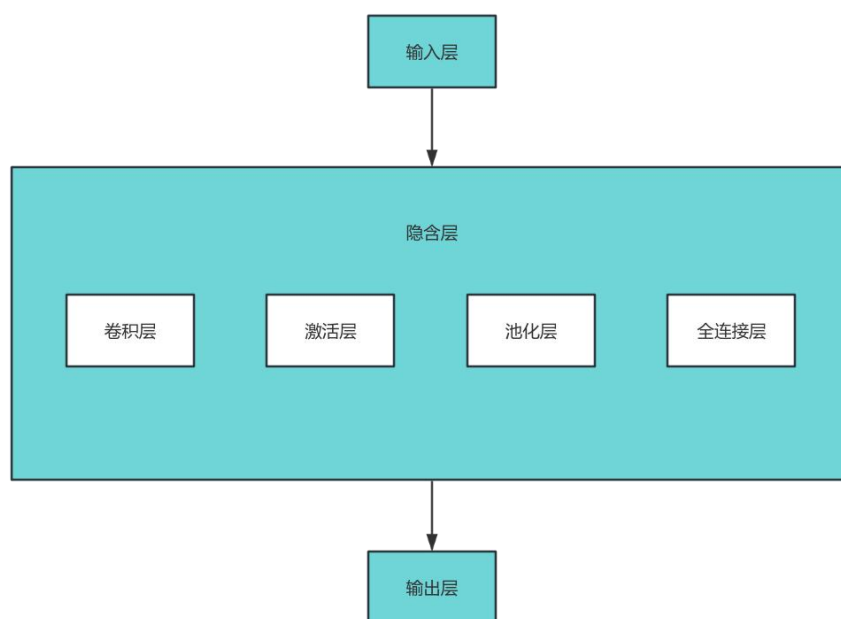


图 2.7 卷积神经网络结构图

卷积层是卷积神经网络的核心模块。卷积核是卷积层中的核心参数，其本质上就是一个多维矩阵。各个卷积核凭借其独特的权重设置，能够十分精准地捕捉到输入数据里的特定模式。与此同时，多个卷积核开展并行运算能够提取出多种多样且丰富的特征表示形式。在具体的运算环节里，卷积核会在输入数据之上缓缓滑动起来，它会将输入数据的对应区域与对应权值相乘，在这之后再进行求和操作，通过这种数学计算操作便能够生成一个全新的数值去体现这片区域的相关特征。当针对整个输入数据完成特征提取的工作之后，所得到的特征会依照输入数据的区域分布情况进行组合最终形成输入数据的特征图，卷积层提取特征流程如图 2.8 所示。这种具备局部连接以及权值共享特点的机制，让卷积层在极大程度上减少参数量的状况下，依旧可以有效地将数据的空间特征关系保留下来。

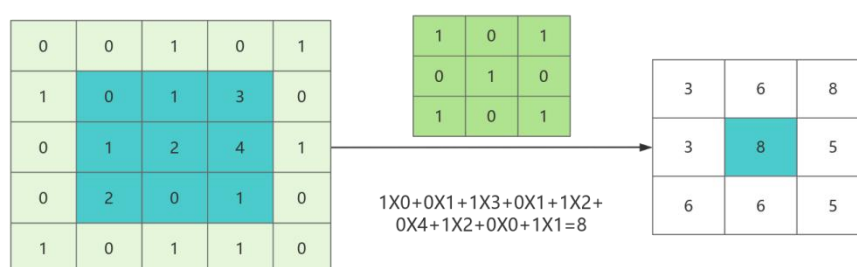


图 2.8 卷积层提取特征

池化层是卷积神经网络里的关键组件，它能够对模型结构予以优化，还可以提升模型的泛化能力。其运作方式是凭借一个池化窗口在输入数据的特征图上面进行移动，然后依照特定的规律去生成某一特定的数值，而这个数值是用来代表该部分区域的特征图的。池化层主要存在两种类型，即最大池化以及平均池化。

在实施最大池化操作时，池化窗口会挑出窗口内最大数值的元素，将其当作输出内容，而其余的那些元素就都被丢弃掉。在实际处理一张带有图像边缘信息的特征图时，最大池化操作能够很精准地捕捉到每一个局部区域当中最强有力的响应信号，并且会以高对比度的形式把边缘细节给保留下来，具体操作如图 2.9 所示。这种处理的方式，一方面有效地让数据维度得以降低，另一方面，这种处理方式也能将特征图里的关键特征给凸显出来，极大地提升模型对于重要信息的敏感程度。

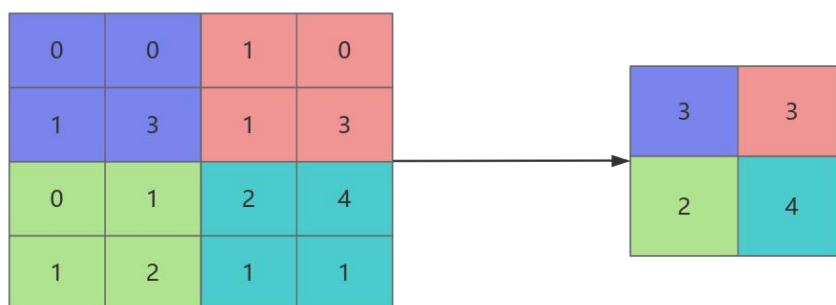


图 2.9 最大池化层

平均池化会把池化窗口内全部元素的平均值计算出来并以此作为输出结果，具体操作如图 2.10 所示。在输入特征图中含有较多细微波动或者存在一些非关键细节的场景下，平均池化借助求均值的办法能够对这些波动进行平缓化处理，以此来保留特征的整体分布态势，通过这种减小局部差异带来影响的操作，平均池化能够较为有效地对局部噪声干扰予以抑制。

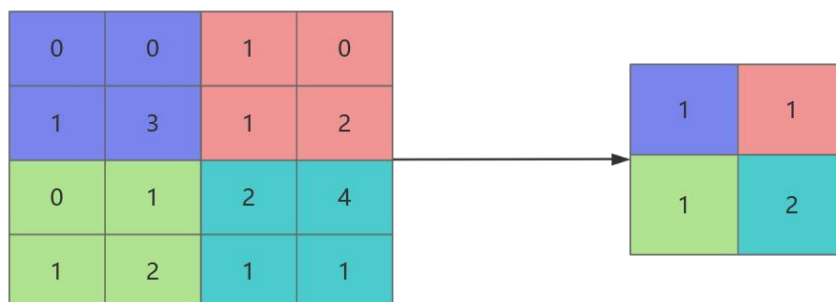


图 2.10 平均池化层

全连接层在从特征提取到决策的过程中属于极为关键的一个环节。当输入的图像按顺序先后经过卷积层以及池化层的时候，网络会逐步地提取出图像里面所包含的边缘、纹理这类局部特征，同时也会提取出像物体部件、空间关系等这样较为抽象的语义特征。这时，网络对于图像的理解方式已经从最初单纯的原始像素值转变成具备层次结构的那种特征表达形式。不过，这种特征表达在结构方面依旧存在着一定的局限性。虽然卷积操作能够比较有效地捕捉到局部的相关信息，可以对特征图中元素的空间位置以及通道维度方面的信息进行保留，但是这样一来，这种操作就使得不同区域以及不同类型的特征之间缺少了直接进行交互和整合的条件。这种呈现出分散状态的特征表达尽管已较为详尽地对图像的各个局部都做出描述，但是依然很难形成对整体图像的统一理解，因此也就无法直接用于判断图像具体所属的类别或者用于预测与之相关的数值。

全连接层关键的功能在于打破空间以及通道的独立性。它会将多维特征图降维成一维向量，在这之后，全连接层构建的连接网络，该网络把降维处理后所得到的向量里的每一个元素都和后续层中的每一个神经元逐个建立起连接，该连接过程如图 2.11 所示。这样的连接的方式不仅能够确保每一个输入特征都可以和所有的输出神经元进行交互，还实现对特征的全局整合以及重组。在这个过程中，全连接层能够充分考虑图像各个部分的特征信息，并且其还能凭借学习特征之间复杂的非线性关系生成输入图像的整体性描述。这种全局表征和模型进行分类或者回归等决策任务更为契合，由此也就实现从特征提取迈向实际应用的重要跨越^[14]。

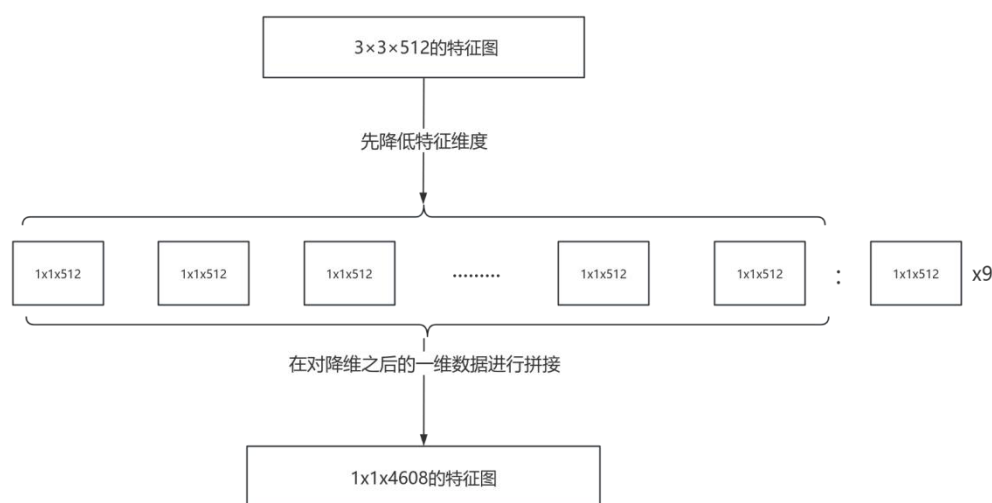


图 2.11 全连接层原理图

在连接环节里，全连接层的每一处连接均配置与之对应的权重参数，这些参数会在训练进程当中借助反向传播算法不停地加以调整以及优化。通过这样的密集连接机制，全连接层是能够把之前经由卷积操作以及池化操作所提取出来的局部特征予以整合的，还能够去挖掘特征相互之间存在的高阶关联，进而构建起那种能够反映数据全局语义的特征表示。最终，全连接层所输出的结果将会成为模型去做出最终决策的直接依据，它能够决定数据所属的类别或者预测数值的大小，成为连接特征学习以及实际应用目标的极为关键的纽带，对于模型的性能表现起着决定性的作用。

2.4 算法分析

在本次人员离岗检测系统中，需要完成对于人员目标的检测以及人员目标的跟踪，为了满足该系统的功能需求，本次系统在算法方面主要运用目标检测算法以及目标跟踪算法。

2.4.1 目标检测算法

目标检测算法在人员离岗行为检测系统中有着极为关键的作用。其负责从采集图像中精准地把人员目标给识别出来，在此过程中，系统还要明确检测目标所处的位置。这一技术融合了模式识别、机器学习以及深度学习等学科的知识，其应用场景颇为广泛。随着人工智能技术不断地发展，目标检测算法也一直在持续不断地进行迭代升级，目的就是为了能够去应对更为复杂的实际需求情况。目标检测算法发展历程大体上可划分为两个阶段，即传统方法阶段以及深度学习方法阶段。

在传统方法阶段当中，目标检测工作主要依靠人工设计出来的特征来开展，经典的目标检测算法如 Haar 特征、HOG 特征等等皆是如此，并且还需与 Adaboost、SVM 等分类器相互结合，以此才能达成目标检测的目的。此阶段所运用的方法主要是先手动去提取图像的相关特征，随后再运用分类器针对候选区域展开分类判断。不过，因为人工设计出来的特征在表达能力方面存在局限，使得最终所得到的检测结果的精度以及泛化能力并不理想，而且该过程的计算复杂度较高，在处理处于复杂场景之下的目标检测任务时，往往出现检测效果偏低的现象，很难满足项目的检测效果需求。

在深度学习逐步兴起的背景下，目标检测领域进入深度学习方法所主导的阶段。与传统的目标检测方法相比较，基于深度学习的目标检测算法能够凭借卷积神经网络从

相关的数据里面去自动地学习相应的特征，除此之外，这种检测算法还能借助多层的神经元一步步地提取出抽象的特征，在很大程度上提升检测结果的精度，同时其泛化能力也得到明显的增强^[15]。基于深度学习的目标检测算法依照在进行检测的过程中是否会生成候选区域可大致划分为两阶段检测算法（Two-stage）以及一阶段检测算法（One-stage）两种类型^[16]。

两阶段算法一开始会借助区域建议网络来生成一连串有可能包含目标的候选区域，算法会针对这些候选区域展开分类操作以及边界框回归操作，这样就能明确目标的类别以及其精确的位置所在，该类算法流程如图 2.12 所示。较为经典的算法有 R-CNN、Fast R-CNN、Faster R-CNN 等。R-CNN 首次把深度学习引入到目标检测领域当中，它采用选择性搜索算法来生成候选区域，把每一个候选区域都输入到卷积神经网络进行特征提取，最后再用 SVM 来完成分类的工作，不过它的计算效率较低，检测速度也缓慢。Fast R-CNN 对 R-CNN 做出一些改进，通过在卷积层进行共享计算使得检测速度有所提升；Faster R-CNN 则是更进一步引入 RPN，用它取代了选择性搜索算法，如此便实现从端到端的训练以及检测，极大地提升算法整体的检测效率还有精度。整体而言，两阶段检测算法的检测精度较高，在处理复杂场景下的小目标、重叠目标等场景时，算法的检测结果较好，由于这种算法需要生成候选区域，这导致算法的检测速度相对来说较慢，难以满足实时性要求较高的场景。

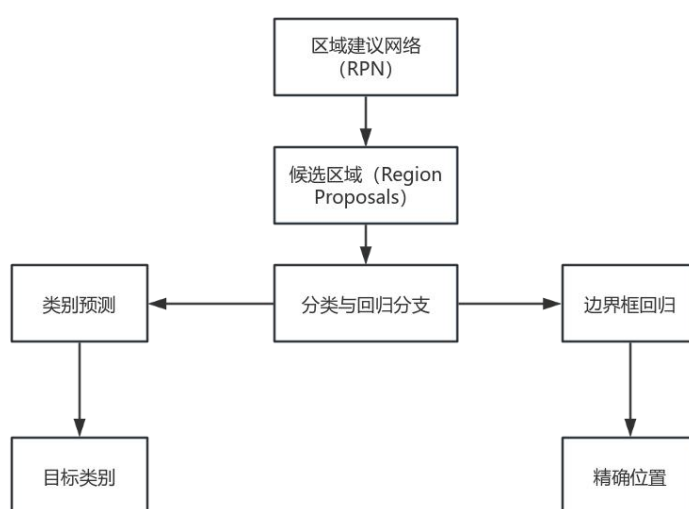


图 2.12 Two-stage 检测算法部分流程

和两阶段算法相比，一阶段检测算法并不需要生成候选区域而是把目标检测任务简化成回归问题，这种算法只通过一次卷积神经网络的前向传播就能直接对目标的类别以及边界框坐标做出预测。这样的设计让检测速度有了明显的提升，能够满足对实时性要求比较高的应用场景。SSD 算法、YOLO 算法等都属于比较有代表性的算法。SSD 在检测的时候采用不同尺度的特征图，它对于不同大小目标的识别就能够更有效，在精度和速度方面实现较好的平衡。YOLO 算法则是将输入图像划分成多个网格，每个网格负责对落入其中的目标进行预测，它仅依靠单次卷积神经网络前向传播就可以完成检测，提高算法的检测速率。YOLO 系列算法还在不断地进行优化，通过引入多尺度特征融合、对网络结构加以改进等方式，逐步地对检测精度进行提升，该算法的流程如图 2.13 所示。

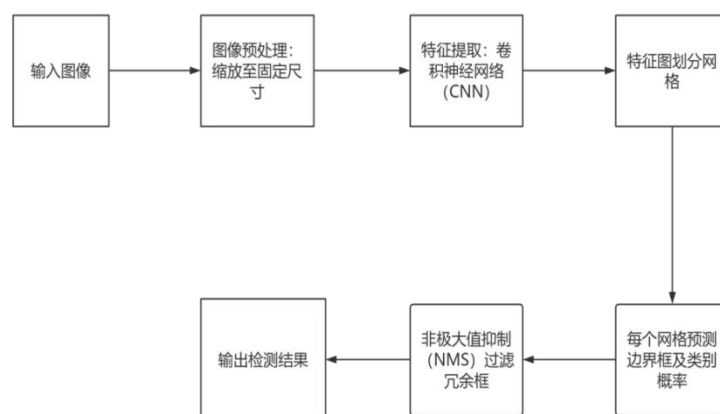


图 2.13 YOLO 检测算法的流程

在 YOLO 目标检测算法把输入图像划分成多个网格的过程中，每一个网格都肩负着对落在自身网格内目标的边界框以及其所属类别进行预测的任务。具体来讲，每个网格会给出边界框的预测值，这里每个边界框都涵盖坐标参数以及置信度得分，置信度得分代表在边界框内存在目标的概率也反映了预测框和真实框之间的重叠程度。除此之外，每个网格还会输出类别概率，这些概率能够体现出该网格内目标属于不同类别的可能性大小。在推理阶段时，算法会借助非极大值抑制的方式来对所有网格输出的边界框加以处理，将置信度较低以及重叠度偏高的冗余框给过滤掉仅保留最为准确的目标检测结果。这种端到端的设计模式不仅减少了计算量，还避开传统两阶段算法里生成候选区域所耗费时间的步骤，使得 YOLO 在能够维持较高检测精度的同时，还能在检测效率方面有显著的提升。

YOLOv5 属于 YOLO 系列检测算法里的一个版本，跟之前同系列的算法比起来，它对骨干网络以及颈部结构都做优化处理，其引入自适应锚框计算的方式以及图片缩放方面的机制。在确保检测精度不降低的情况下，模型训练的效率以及推理速度都得到很大幅度的提升。和后续版本的检测算法相比，YOLOv5 在移植的灵活性、社区生态状况还有轻量化设计等方面依旧有着相当显著的优势。另外，YOLOv5 还给出多类型的轻量化模型，当将模型移植到开发板以后，对于硬件的参数需求比较低，模型在移植后更容易高效以及稳定的运行^[17]。

目标检测算法所涉及的评价指标，大体上涵盖准确率、召回率、平均精度均值以及检测速度等方面。准确率所体现检测正确的目标数量在总检测目标数量当中所占的比例，其主要作用在于对算法检测结果的正确性予以衡量；召回率呈现的是检测正确的目标数量于实际目标数量里所占的比例，这能够反映出算法针对目标的检测能力如何；mAP 则是综合考量不同类别目标在各异召回率情形下的平均精度。检测速度通常借助每秒检测帧数或者每张图像的检测时间来加以衡量，在实时性要求较高的应用场景中，检测速度的重要性是不言而喻的。

2.4.2 跟踪算法

跟踪技术是人员离岗监测系统里的关键算法，它最为突出的核心价值体现在能够很好地应对那些处在复杂场景之下对目标持续追踪的场景。在监控场景中出现目标被遮挡、出现快速位移或者目标外观发生变化时，相应的跟踪算法可以稳定地锁定人员目标，以此来确保监测工作能够保持连续性。依据技术演进与实现原理，当前目标跟踪算法可划分为基于传统方法和基于深度学习方法两大类。

传统跟踪技术以卡尔曼滤波以及粒子滤波作为典型代表，其构建的基础是概率统计理论。就卡尔曼滤波来讲，它借助线性系统假设这一条件建立了预测与校正的双阶段的机制，在噪声呈现出符合高斯分布的状况下，其表现相当出色。而粒子滤波则是采用蒙特卡洛采样的策略，通过利用数量极为庞大的随机粒子来对目标状态的概率分布进行逼近，它便能够适应存在非线性运动的场景。这一类的方法凭借着在计算效率方面所具有的优势在以往的相关领域占据着主流的地位。不过，当需要去应对外观出现形变、长时间存在遮挡等较为复杂的工况的时候，这类方法所存在的精度衰减方面的问题就逐渐地凸显出来，也正是因为如此，促使整个行业开始朝着深度学习技术路

径的方向去转变。

随着深度学习在计算机视觉这一领域被广泛加以应用，基于深度学习的目标跟踪算法也随之取得快速的发展。借助卷积神经网络所具备的特征提取能力，跟踪方法能够去学习目标所具有的深层语义表征。从技术架构方面来看，深度跟踪方案分化成单目标跟踪以及多目标跟踪两种方法^[18]。

单目标跟踪算法着重于对视频序列里的单个目标展开持续不断的跟踪操作。常见的 Siamese 网络系列算法有着独特的孪生网络构造，会把目标模板以及待检测帧都输入到网络当中，通过去计算这两者之间的相似度，进而确定出目标所在的位置。像 SiamFC、SiamRPN 等都属于典型的这类算法，它们凭借在大规模数据集上预先开展训练活动，以此来学习目标具有的通用特征，在跟踪的进程中能够较快速地适应目标外观方面出现的变化，从而有效地提升算法的跟踪效果。

多目标跟踪算法的主要任务是要在视频序列当中对多个目标同时展开跟踪操作，通过给每一个目标都分配上独一无二的标识，进而生成出其对应的运动轨迹来。这类算法一般情况下，会把检测以及跟踪这两个模块相结合。先是借助目标检测算法去得到目标所在的位置，在这之后，再依靠数据关联算法把处于不同帧里面的目标完成匹配的工作。SORT、DeepSORT 都是有代表性的算法，它们通过把目标的外观特征和运动信息融合起来的方式，较为有效地处理目标被遮挡、发生交叉等等这些复杂的状况，最终实现对多目标进行准确跟踪的目的。

SORT 作为多目标跟踪领域的经典算法，凭借其轻量高效的特性，该算法在实时视频分析、智能监控等场景中占据重要地位。该算法依照先检测后跟踪技术逻辑来开展相关操作，它会对 YOLO 等目标检测模型加以整合，针对视频流展开逐帧的细致解析，从中提取出目标的位置坐标，还有类别属性以及检测置信度等方面的信息，从而为后续的追踪流程给予基础数据方面的有力支撑。

在运动预测方面，SORT 运用卡尔曼滤波来搭建线性运动模型。它通过定义一个包含着位置、速度等诸多参数的状态向量使得算法能够针对目标在下一帧有可能出现的潜在位置展开相应的数学建模操作。在数据关联这个环节当中，算法运用几何匹配的策略，通过对预测框和检测框的交并比来达成目标身份匹配。当两者的重叠区域超出预先设定的阈值时，那么系统便会判定这两个框属于同一个目标，进而完成轨迹的更新操作；若某预测框连续多帧未匹配成功，则认为目标已离开视野并删除对应轨迹，

新出现的检测框将被初始化为新的跟踪轨迹。这样一种颇为简洁的匹配机制，让 SORT 既能维持住计算方面的效率，又能够在常规的场景之下有效地去应对多目标跟踪所产生的种种需求。

DeepSORT 在 SORT 算法的基础上进行改进^[19]。一开始，会借助目标检测算法来获取目标所处的位置，接着再通过卡尔曼滤波去预测目标在接下来一帧当中的位置情况。在数据关联阶段，DeepSORT 不仅考虑目标的运动信息，还提取目标的深度外观特征，通过计算外观特征的余弦距离和马氏距离进行目标匹配。这种多信息融合的方式，使 DeepSORT 在处理目标遮挡、相似目标干扰等复杂场景时，能够更准确地维持目标的身份标识，生成连续、稳定的运动轨迹，图 2.14 为该算法流程图。

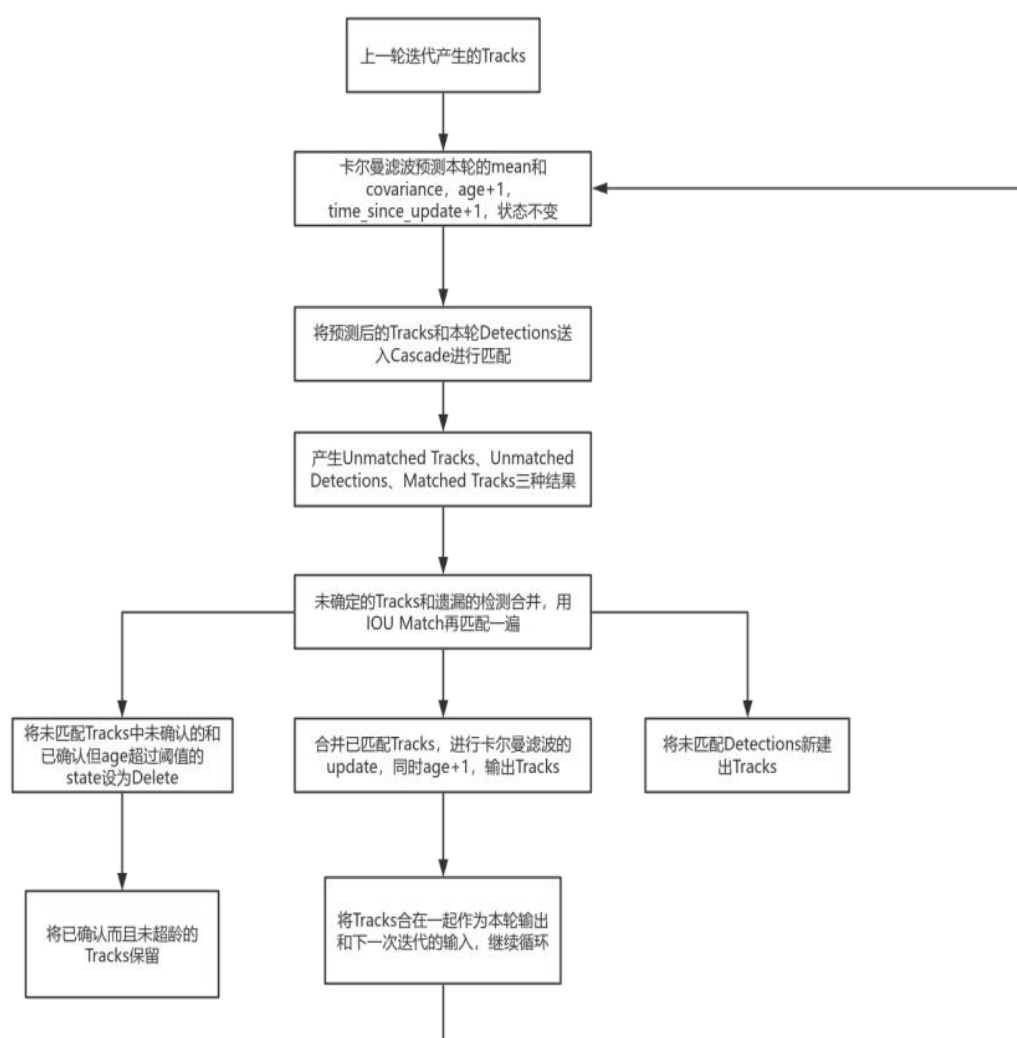


图 2.14 DeepSORT 跟踪算法原理

3 系统设计与实现

此次系统构建主要围绕人员检测模型搭建、跟踪算法的接入、人员检测模型转换以及检测模型的开发板移植四个方面来展开。人员检测模型搭建方面主要包含神经网络模型、数据集处理以及人员检测模型训练模型三个部分内容。跟踪算法的接入跟踪算法的接入主要介绍 DeepSORT 算法的本地实现原理以及在代码中的调用过程。而人员检测模型转换部分由把 Pytorch 模型转换为 ONNX 模型以及将 ONNX 模型进一步转成 RKNN 模型这两部分所构成。至于检测模型的开发板移植部分主要分为开发板环境搭建和对开发板环境以及模型移植效果的测试这两部分，系统设计与实现结构安排如图 3.1 所示。

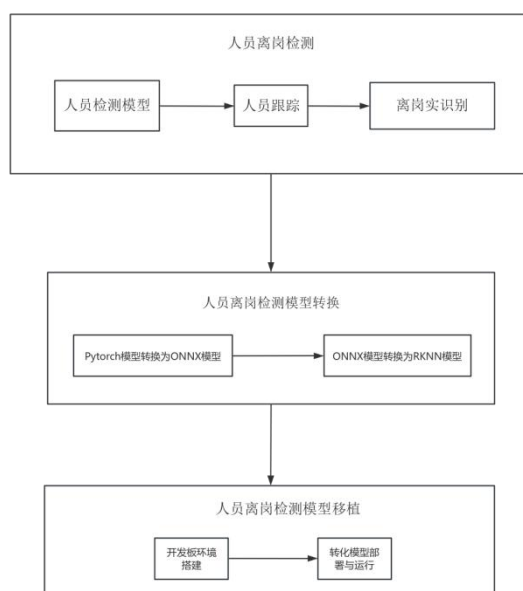


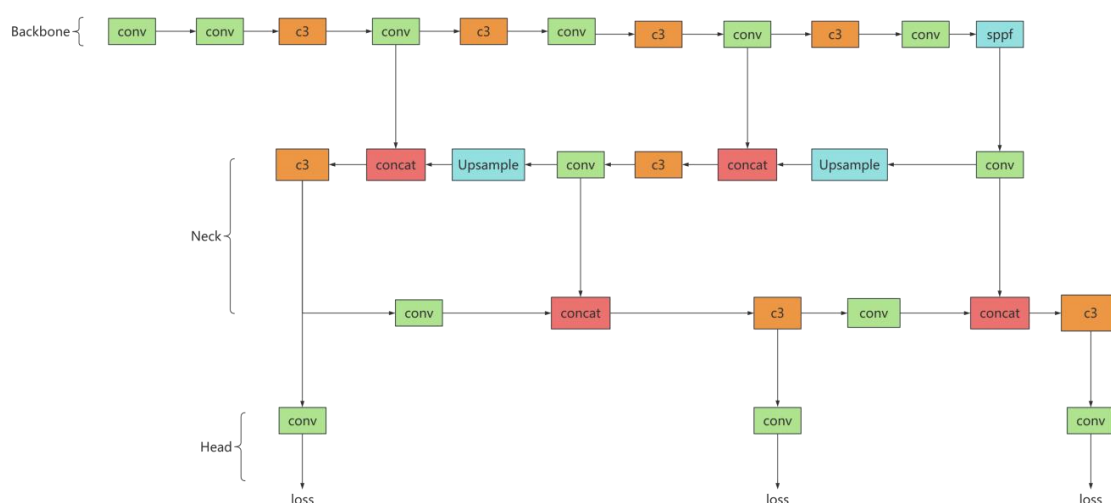
图 3.1 总体设计流程图

3.1 人员检测模型及离岗检测算法设计

对于检测模型的搭建主要在 Windows 操作系统中进行。所用到的主要工具是 2022 版本的 pycharm，凭借该软件下载第三方库方便以及项目环境易于更改的特性，使得在本地开展的模型训练以及训练所达成的效果能够更加直观地呈现出来，也更加便于后期对于模型的效果参数的改进。

3.1.1 人员检测模型

此次设计运用 YOLOv5 架构来搭建人员检测系统，其模型结构如图 3.2 所示。该网络呈现出三级递进式的架构形式：特征提取层（Backbone）运用跨阶段部分连接结构即 CSP，借助梯度流分阶段融合策略，一方面能够对计算量加以压缩，另一方面还可以提升特征表征的能力，不但能保证系统实现轻量化运行，而且还能强化对于图像细节的感知程度；特征融合层（Neck）颇具创新性地采用 FPN+PAN 双路径机制，从顶部往底部传递高层语义方面的信息，同时从底部往顶部融合底层定位的相关细节，由此构建出双向信息流通道，让特征图既拥有对目标类别进行判断的能力，又具备空间坐标感知的精准度；预测输出层（Head）配备了三尺度检测头，同步生成 48×48 、 96×96 、 1536×1536 像素级别的特征图，每个网格单元都能够独立去预测 4 个边界框以及与之对应的类别概率，通过对多层级特征进行解耦操作，进而实现对全尺度目标的覆盖。这样的设计在维持比较高的检测速度之际，还十分明显地提升不同尺寸目标的平均检测精度，特别是在资源受到一定限制的嵌入式场景当中，很好地展现出检测精度与推理效率之间那种完美的平衡状态。



让目标区域的特征响应得到明显的强化。这样的双阶段注意力校准策略能使模型在复杂的场景当中可以自动聚焦在人员目标的特征区域上，特别是在监控场景里针对小目标进行检测的时候，添加注意力机制后的目标检测效果得到极大的提升。该特征增强机制一方面增强了模型的鲁棒性，另一方面还借助其动态特征重校准的能力，保证模型的环境适应能力，为人员监测系统的稳定运行提供技术支持。

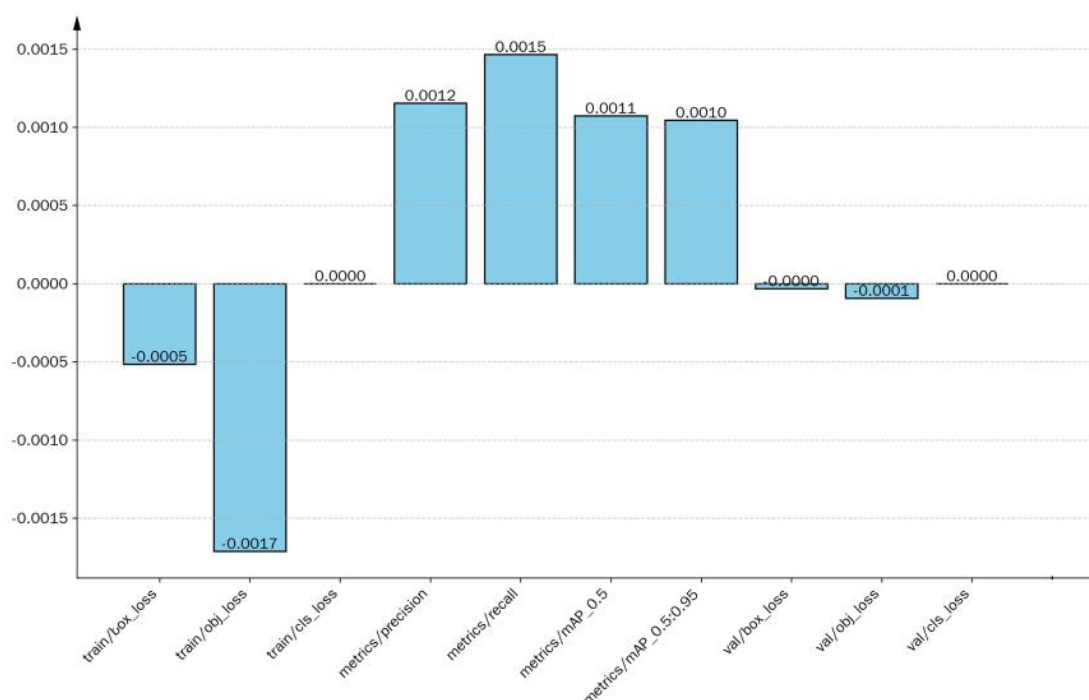


图 3.3 添加注意力机制后的模型效果提升

此次所运用的数据集为 CrowdHuman 数据集，数据集大概有着 470,000 个人体实例，其涉及的场景十分丰富多样，人群密度情况也各有不同。它给出详尽的标注信息，像头部边界框、可见区域边界框以及全身体边界框等数据，能更为有效地模拟出实际场景所具有的复杂性与多样性。

对于该数据集的处理主要是对其标签文件进行处理，当我们直接下载完成的该数据是 ODGT 格式文件，该格式的标注文件每行代表一个以 JSON 格式存储的图像的标注信息并不能直接用于 YOLO 模型的训练。因此我们需要先行对标签文件进行处理。该过程首先通过代码读取标签文件后，在获取到标签文件之后通过调用 split 函数对其切分，以获取标签文件对应的图片名称、检测框的位置以及检测物体的类型，然后在通过标签文件检索对应的图片文件，读取该图片的真实尺寸以用于后面的检测框坐标的归一化处理，在归一化处理完成之后，按照类别加左上角坐标和右下角坐标

的方式存储，该过程的流程如图 3.4 所示

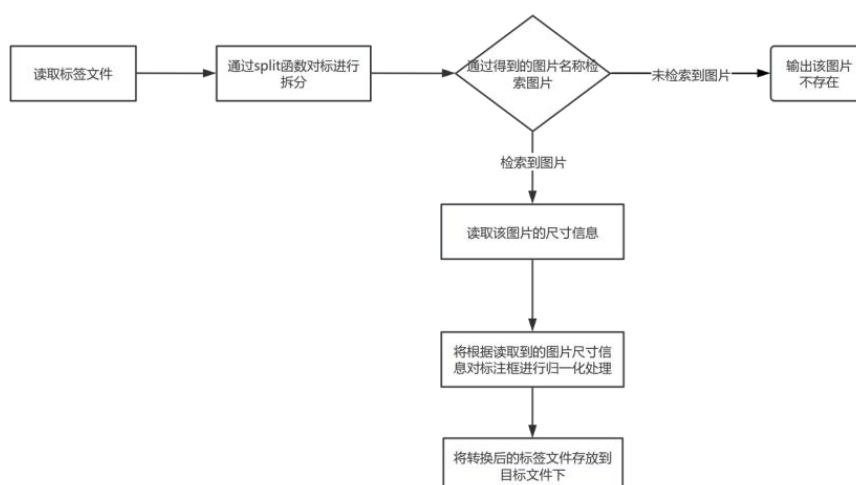


图 3.4 数据集标签文件处理

在整体过程中，代码需要指定转换之后的标签文件输出地址、代转换文件以及图片文件。代码会从指定的图片目录中获取所有图片文件并按名称排序，之后根据图片文件和标注数据的数量确定可处理的最小数量以此进行后续处理，避免因数据不匹配导致的错误。代码会对每张图片进行重命名操作。然后遍历每张图片的标注数据，提取标注数据中的坐标信息过滤掉无效的标注，计算中心坐标、宽度和高度的归一化值，标签文件转化后如图 3.5 所示。

data	1	p	0.852051	0.558824	0.058886	0.138515
runs	2	0	0.811035	0.565978	0.034180	0.139905
success	3	0	0.823242	0.517488	0.023438	0.077901
run	4	0	0.806641	0.518283	0.017578	0.079491
run1	5	0	0.792480	0.529412	0.018555	0.079491
run3	6	0	0.775391	0.519873	0.015625	0.069952
run4	7	0	0.767578	0.540541	0.019531	0.069952
images	8	0	0.749023	0.553259	0.021484	0.079491
labels	9	0	0.671875	0.666932	0.126953	0.453180
train	10	0	0.625000	0.647059	0.128906	0.467409
visualization	11	0	0.551758	0.570747	0.041016	0.209318
273271_1a2786000b0c7a077.txt	12	0	0.535156	0.652623	0.074219	0.402226
273271_1a02000084ad5ae8.txt	13	0	0.516602	0.589025	0.064453	0.356121
273271_1a51000317893a5.txt	14	0	0.465820	0.666137	0.093759	0.416534
273271_1ac00092ad10cd.txt	15	0	0.478516	0.610493	0.072266	0.352941
273271_1b9eb0008049cd6.txt	16	0	0.416992	0.745628	0.195312	0.416534
273271_1b86000bc5b77bf.txt	17	0	0.418457	0.627981	0.083008	0.349762
273271_1b976000f70236c1.txt	18	0	0.388356	0.692369	0.091797	0.373609
273271_1ba93000e0e011c.txt	19	0	0.388371	0.619237	0.091727	0.392607
273271_1ba470008722b054.txt	20	0	0.326172	0.662162	0.109375	0.456280
273271_1ba0200041e0121f.txt	21	0	0.315918	0.709897	0.110352	0.383148
273271_1ba400001d13c4b.txt	22	0	0.269043	0.674086	0.098633	0.419714
273271_1bc75000ad1a58db.txt	23	0	0.308594	0.646264	0.070312	0.376709
273271_1bcf6000b9a2c34e.txt	24	0	0.243164	0.649444	0.072266	0.421304
273271_1bdb20008371af00.txt	25	0	0.198730	0.663752	0.122070	0.427643
273271_1bfe9000ca139ae5.txt	26	0	0.166016	0.674881	0.095703	0.456280
273271_1c02a00058d9f56e.txt						
273271_1c3a7000ee250436.txt						
273271_1c3c10003f6420d9.txt						

图 3. 转换后的标签文件

在 PyTorch 框架下展开模型训练相关事宜，其主要涵盖模型训练代码的编写工作以及数据导入方面的内容。模型训练代码借助对训练效果参数、训练轮数以及学习率加以设置的方式，来促使模型的训练效果得以提升。在此次设计的模型训练阶段运用预训练模型，这种方法能够缩减训练时间，并且让训练效果更快地达到收敛状态。另

外，模型训练的部分代码可参照图 3.6。

```
callbacks.run('on_train_end', last, best, plots, epoch, results)

torch.cuda.empty_cache()
return results

def parse_opt(known=False):
    parser = argparse.ArgumentParser()
    parser.add_argument('--weights', type=str, default=ROOT / 'yolov5s.pt', help='initial weights path')
    parser.add_argument('--cfg', type=str, default='', help='D:\Desktop\毕业设计\人员在离岗识别\项目代码\models\yolov5s.yaml')
    parser.add_argument('--data', type=str, default=ROOT / 'data/coco128.yaml', help='dataset.yaml path')
    parser.add_argument('--hyp', type=str, default=ROOT / 'data/hyps/hyp.scratch-low.yaml', help='hyperparameters path')
    parser.add_argument('--epochs', type=int, default=200)
    parser.add_argument('--batch-size', type=int, default=16, help='total batch size for all GPUs, -1 for autobatch')
    parser.add_argument('--imgsz', '--img', '--img-size', type=int, default=640, help='train, val image size (pixels)')
    parser.add_argument('--rect', action='store_true', help='rectangular training')
    parser.add_argument('--resume', nargs='?', const=True, default=False, help='resume most recent training')
    parser.add_argument('--nosave', action='store_true', help='only save final checkpoint')
    parser.add_argument('--noval', action='store_true', help='only validate final epoch')
    parser.add_argument('--noautoanchor', action='store_true', help='disable AutoAnchor')
```

图 3.6 部分训练模型代码

数据导入这一环节，重点在于创建一个 yaml 文件来设定训练数据的路径。借助创建此文件的操作，我们能够较为便捷地对模型的训练数据来源做出快速修改，进而让模型的训练变得更为便利。在该文件的数据当中，存有检测类别的数量以及相应的名称，本次模型训练所用的创建的文件内容如图 3.7 所示。

```
4 # parent
5 # |— yolov5
6 # |— datasets
7 # |— coco128 ← downloads here (7 MB)
8
9
10 # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, ...
11 path: ../datasets/xun5 # dataset root dir
12 train: images/train # train images (relative to 'path') 128 images
13 val: images/val # val images (relative to 'path') 128 images
14 test: # test images (optional)
15
16 # Classes
17 nc: 1 # number of classes
18 names: ['person'] # class names
19
```

图 3.7 与训练代码对应的 yaml 文件

3.1.2 跟踪算法及离岗检测

在完成检测模型的相关搭建之后需要完成 DeepSORT 算法的本地落实，在该过程中，需要完成对算法代码文件进行获取并且保证代码文件在本地能够正常运行，也就是需要安装改代码文件的依赖库。在最终安装完成之后，我们仅需通过调用该功能带啊吗库即可完成对跟踪算法的调用，具体的跟踪算法声明调用如图 3.8 所示，在实际使用时，仅需传入对应的参数即可。

```

import torch
import cv2
import numpy as np
from deep_sort_realtime.deepsort_tracker import DeepSort
from collections import defaultdict
import time
from ultralytics import YOLO

```

图 3.8 DeepSORT 算法的本地调用

在完成对跟踪算法的本地调用之后还需在代码中对人员的具体位置参数进行传入以完成对人员的跟踪，在本地代码部分对跟踪器更新代码如图 3.9 所示。通过更新跟踪器能够有效的完成对于目标的跟进。

```

# 提取检测到的目标信息
detections = []
for xxyy, conf, cls_id in zip(predictions, confidences, class_ids):
    if conf > 0.5 and model.names[int(cls_id)] == 'person':
        x1, y1, x2, y2 = map(int, xxyy)
        # 关键修改: 使用元组格式 (bbox, confidence)
        detections.append((x1, y1, x2 - x1, y2 - y1, conf)) # DeepSort 需要 [x,

# 使用 DeepSort 进行目标跟踪
tracks = tracker.update_tracks(detections, frame=frame)

```

图 3.9 跟踪器调用以及参数传入

在完成对于跟踪算法的导入之后，需要对于系统的离岗检测算法进行实现。在这部分当中，算法需要对所采集到的图像信息里的人员信息进行针对性的检测，而且在处理同一目标处于不同位置时，算法需要将其视为同一检测目标，本次离岗行为检测系统的流程图如图 3.10 所示。

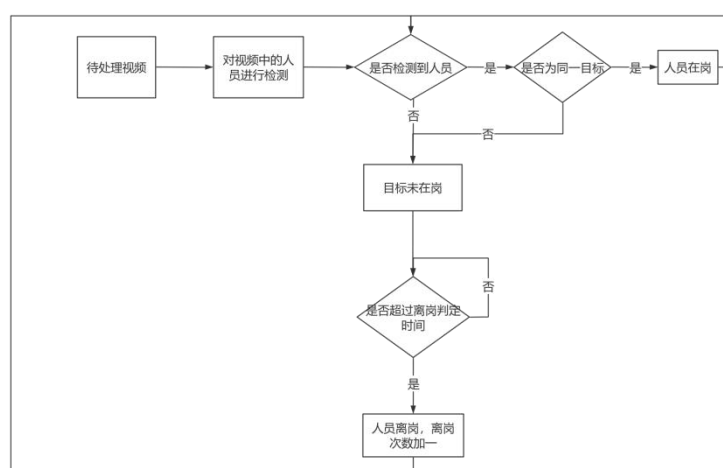


图 3.10 离岗行为检测系统流程图

具体而言，系统对采集的图像中的人员实施检测，在采集图像中检测到人员时视

为人员在岗也就是离岗行为未发生,如果系统在采集的图像中未检测到人员则判定目标为未在岗状态但不记录离岗次数,在未检测到目标的时间超过预先设置的离岗时间时,则记为离岗行为发生,系统记录离岗次数并将离岗结果在视频中展示,图 3.11 所示离岗判定代码。

```
# 检查离岗情况
current_time = time.time()
for track_id, last_seen in list(last_seen_times.items()):
    if current_time - last_seen > leave_time:
        if track_id not in off_post_alerts:
            print(f'人员 person{track_id} 已离岗')
            off_post_alerts.add(track_id)
            off_post_log.append((track_id, time.strftime("%Y-%m-%d %H:%M:%S", t
            log_file.write(f'person{track_id}, {time.strftime("%Y-%m-%d %H:%M:%S", t
        else:
            off_post_alerts.discard(track_id)
```

图 3.11 离岗判定代码

3.2 人员离岗检测模型转换

模型转换属于开发板移植之前极为关键的一个步骤,其在提升模型于开发板之上的运行效率以及性能方面有着不可忽视的重要作用。合理地去运用模型量化、压缩之类的相关技术再结合专业的模型转换工具的同时在开展严格的测试验证工作,能够保证模型在开发板上可以稳定且高效地运行起来。此次模型的转换流程是在 Linux 子系统当中开展的,所采用的系统镜像是 Ubuntu 22.04.5 LTS。Linux 子系统依靠自身所具备的独特优势,给模型转换的整个过程带来许多的便利条件。它那设计颇为精巧的文件系统架构,能够让子系统当中的文件和主操作系统的文件达成无缝共享的状态,并且还可以很便捷地对这些文件进行更改操作。

本次模型转换在 Visual Studio Code 软件中进行。通过安装 Visual Studio Code 所给予的 WSL 扩展,我们能够在本地系统下对 Linux 子系统里面的文件展开查看和修改操作。在子系统里安装好相应版本的 miniconda 之后,便可对转换工具所需要的依赖库进行安装。这些依赖库为模型转换工具的正常运行给予相应的环境方面的支撑,保证转换的整个过程是可以顺利地完成的。在所有的依赖库都安装完成以后,就可以正式开启模型格式的转换工作。依靠子系统在文件访问方面所具备的便利性以及软件所给予的开发方面的支持,本地开展模型转换的这个过程变得更为简便。

3.2.1 PyTorch 模型转换为 ONNX 模型

在 PyTorch 模型朝着 RKNN 格式去做转换的时候,ONNX 中间格式起到极为关

键的一种过渡作用。ONNX 它是通过把 PyTorch 的那种动态计算图转变成为静态的表示形式，以此达成在不同框架之间实现解耦以及达到标准化这样的效果。要知道，RKNN-Toolkit 工具没办法直接就把 PyTorch 模型给转换成 RKNN 模型，所以我们得先把 PyTorch 模型转变成 ONNX 模型。而且，ONNX 还对动态轴定义予以支持，能够很有效地去处理模型的 batch size 或者输入尺寸这些方面，从而为 RKNN 的静态编译提供必不可少的相关信息。

在将 PyTorch 模型转变成 ONNX 框架时，首先得对在本地下载好的 YOLO 文件做出一些必要的修改操作。要先确定文件所在的目录，然后定位到该目录下的模型文件夹，从中找出那个 yolo.py 文件。这个 yolo.py 文件里涵盖 YOLO 模型最为核心的架构方面的定义内容，也包含其前向传播的相关逻辑，还带有后处理模块，它在整个模型转换的过程中可是起着极为关键的作用。关于对 yolo.py 文件的修改情况，可参照图 3.12。与此同时，还需要针对 export.py 文件去做一部分的修改工作，其修改情况如图 3.13 所示。

```
#         if isinstance(self, Segment): # (boxes + masks)
#             xy, wh, conf, mask = x[i].split((2, 2, self.nc + 1, self.no - self.nc - 5), 4)
#             xy = (xy.sigmoid() * 2 + self.grid[i]) * self.stride[i] # xy
#             wh = (wh.sigmoid() * 2) ** 2 * self.anchor_grid[i] # wh
#             y = torch.cat((xy, wh, conf.sigmoid(), mask), 4)
#         else: # Detect (boxes only)
#             xy, wh, conf = x[i].sigmoid().split((2, 2, self.nc + 1), 4)
#             xy = (xy * 2 + self.grid[i]) * self.stride[i] # xy
#             wh = (wh * 2) ** 2 * self.anchor_grid[i] # wh
#             y = torch.cat((xy, wh, conf), 4)
#         z.append(y.view(bs, self.na * nx * ny, self.no))

#     return x if self.training else (torch.cat(z, 1),) if self.export else (torch.cat(z, 1), x)

def forward(self, x):
    z = [] # inference output
    for i in range(self.nl):
        x[i] = self.m[i](x[i]) # conv
    return x
```

图 3.12 yolo.py 文件 forward 函数修改

```
im, model = im.half(), model.half() # to FP16
# shape = tuple((y[0] if isinstance(y, tuple) else y).shape) # model output
shape = tuple(y[0].shape)
metadata = {'stride': int(max(model.stride)), 'names': model.names, 'max'
```

图 3.13 export.py 文件局部修改

在完成对文件的修改操作之后，需要将所需转换的模型文件逐一复制到主文件目录之中。在终端这里传入相应的转换模型，并且对该模型的输入张量进行相应的设置。这个模型就能够从 Pytorch 模型顺利转变成 ONNX 模型，图 3.14 所示为本次模型的转换结果。

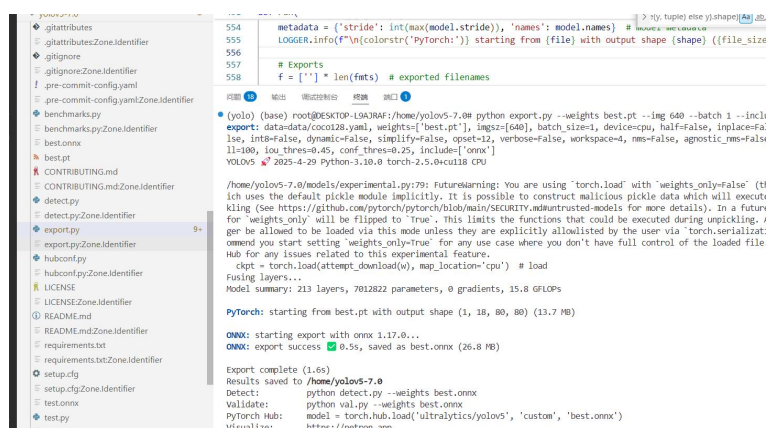


图 3.14 Pytorch 模型转 ONNX 模型转换结果

3.2.2 ONNX 模型转换为 RKNN 模型

在 ONNX 模型朝着 RKNN 格式去进行转换的整个流程里面，RKNN-Toolkit 可是起到极为关键的核心引擎作用。它会针对 ONNX 模型展开图优化以及算子映射方面的相关操作，便达成从标准中间表示到硬件专用格式的这种转换。RKNN-Toolkit 还会对 ONNX 模型开展逐层的解析以及重构方面的工作，通过这样的方式保证了模型在瑞芯微平台上能够实现高效的运行。模型的转换工作主要是依靠 RKNN-Toolkit 这个工具来完成的，一开始的时候，我们得去调用该工具，以便能把要转换的 ONNX 模型成功转换成为 RKNN 模型。在把对应的依赖库都安装好之后，接着就要去安装与之对应的 python 版本的转换工具。此次所采用的是 python3.10 版本的转换工具，与之对应的依赖库情况就如同图 3.15 所示。

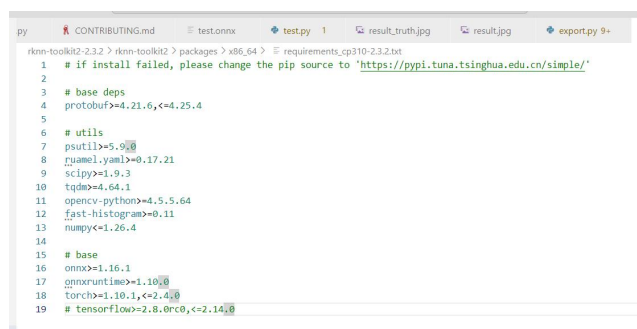


图 3.15 对应的转换工具的依赖库

在对模型予以转换之际，得针对转换文件 test.py 做出特定的修改操作。具体来讲，要依照自己所训练的本地模型的实际情况，对模型的输入图片尺寸加以调整，对输入模型予以适配性修改，并且把生成模型的名模型的检测类别也进行对应的改动，具体修改情况如图 3.16 所示。

```
rknn-toolkit2-2.3.2 > rknn-toolkit2 > examples > onnx > yolov5 > test.py > ...
1 import os
2 import traceback
3 import time
4 import sys
5 import numpy as np
6 import cv2
7 from rknn.api import RKNN
8
9 # Model from RKNN Model Zoo
10 ONNX_MODEL = 'best.onnx'
11 RKNN_MODEL = 'best.rknn'
12 IMG_FOLDER = './images' # 修改为图片文件夹路径
13 DATASET = './dataset.txt'
14
15 QUANTIZE_ON = True
16
17 OBJ_THRESH = 0.25
18 NMS_THRESH = 0.45
19 IMG_SIZE = 640
20
21 CLASSES = ("person")
22
```

图 3.16 模型转换文件的修改

在将转换工具中的转换文件按自己的模型修改之后，即可直接运行转换文件，进行模型的 ONNX 到 RKNN 的格式转换，转换函数运行结果，以及转换后模型的检测结果分别如图 3.17、图 3.18 所示。

```
done
--> Running model for ./images/random_0004.jpg
I GraphPreparing : 100% | 146/146 [00:00<00:00, 19333.47it/s]
I SessionPreparing : 100% | 146/146 [00:00<00:00, 2420.26it/s]
--> Running model for ./images/random_0015.jpg
--> Running model for ./images/random_0001.jpg
--> Running model for ./images/random_0009.jpg
--> Running model for ./images/random_0007.jpg
--> Running model for ./images/random_0008.jpg
--> Running model for ./images/random_0011.jpg
--> Running model for ./images/random_0010.jpg
--> Running model for ./images/random_0016.jpg
--> Running model for ./images/random_0003.jpg
--> Running model for ./images/random_0031.jpg
--> Running model for ./images/random_0005.jpg
--> Running model for ./images/random_0013.jpg
--> Running model for ./images/random_0012.jpg
--> Running model for ./images/random_0002.jpg
--> Running model for ./images/random_0006.jpg
C:\rknn> python3 test.py --onnx ./best.onnx --rknn ./best.rknn --img_folder ./images --dataset ./dataset.txt
```

图 3.17 模型转换函数运行结果



图 3.18 转换模型检测结果

3.3 人员离岗检测模型的开发板移植

3.3.1 开发板环境配置

在开发板环境配置阶段的主要内容为安装模型转换工具的依赖库，确保转换工具在开发板内的正常运行。首先要安装 miniconda，从官网下载好相应版本的 miniconda 之后，重新启动终端接着进入 conda 环境。在这之后，创建一个用来存放模型推理工具的依赖库的 conda 环境，这种操作避免了后续添加其他功能模块时出现版本冲突的情况。在把依赖库下载好之后，再安装与对应 python 版本匹配的模型工具推理，图 3.19 所示内容为模型推理工具安装过程。

```
(myenv) cat@lubancat:~/Desktop/xiong$ ls
rknn-toolkit-lite2  rknn-toolkit-lite2-2.3.2-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
(myenv) cat@lubancat:~/Desktop/xiong$ cd rknn-toolkit-lite2/
(myenv) cat@lubancat:~/Desktop/xiong/rknn-toolkit-lite2$ ls
CHANGELOG.txt  README.md  setup.py
(myenv) cat@lubancat:~/Desktop/xiong/rknn-toolkit-lite2$ cd packages/
(myenv) cat@lubancat:~/Desktop/xiong/rknn-toolkit-lite2/packages$ ls
packages.md5sum
rknn_toolkit_lite2-2.3.2-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp312-cp312-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp37-cp37m-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
(myenv) cat@lubancat:~/Desktop/xiong/rknn-toolkit-lite2/packages$ pip install rknn_toolkit_lite2-2.3.2-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
Processing ./rknn_toolkit_lite2-2.3.2-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
Requirement already satisfied: numpy in /home/cat/miniconda3/envs/myenv/lib/python3.10/site-packages (from rknn_toolkit_lite2==2.3.2)
Requirement already satisfied: psutil in /home/cat/miniconda3/envs/myenv/lib/python3.10/site-packages (from rknn_toolkit_lite2==2.3.2)
Requirement already satisfied: ruamel.yaml in /home/cat/miniconda3/envs/myenv/lib/python3.10/site-packages (from rknn_toolkit_lite2==2.3.2)
Requirement already satisfied: ruamel.yaml.clib>=0.2.7 in /home/cat/miniconda3/envs/myenv/lib/python3.10/site-packages (from ruamel.yaml->rknn_toolkit_lite2==2.3.2)
Installing collected packages: rknn-toolkit-lite2
Successfully installed rknn-toolkit-lite2-2.3.2
(myenv) cat@lubancat:~/Desktop/xiong/rknn-toolkit-lite2/packages$
```

图 3.19 模型推理工具安装

当完成工具的安装工作以后，RK3588 开发板里面内置有诸多不同版本的驱动程序，只是这些驱动程序在默认的情况下全都是处于关闭状态的。基于这样的情况，我们就得打开开发板的设备树文件，然后通过手动操作的方式把与之相对应的摄像头驱动程序给开启如图 3.20 所示。

```
(15)) # cam3
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam3-imx415-overlay.dtb
(16)) #dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam3-imx415-1080p-overlay.dtb
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam3-ov8858-3264x2448-15fps-overlay..
dtbo
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam3-ov8858-1632x1224-30fps-overlay..
dtbo
# cam4
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam4-imx415-overlay.dtb
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam4-imx415-1080p-overlay.dtb
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam4-ov8858-3264x2448-15fps-overlay..
dtbo
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam4-ov8858-1632x1224-30fps-overlay..
dtbo
# cam5
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam5-imx415-overlay.dtb
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam5-imx415-1080p-overlay.dtb
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam5-ov8858-3264x2448-15fps-overlay..
dtbo
#dtoverlay=dtb/overlay/rk3588-lubancat-5io-cam5-ov8858-1632x1224-30fps-overlay..
dtbo
#overlay_end
:wq
```

图 3.20 打开摄像头驱动

3.3.2 开发板环境与模型移植效果测试

在这一环节当中，我们着手编写一个用于检测图片的脚本，以此来达成对转换模型的移植效果进行测试的目的，同时也能够对其环境搭建所产生的结果予以检测。检测代码主要涵盖导入模型以及那些有待检测的图片、输出相关的结果以及获取相应的图片尺寸方面的信息并且对其做后续的处理工作。部分代码如图 3.21 所示。

```

4  import numpy as np
5  from rknnlite.api import RKNNLite
6
7  # 模型和图片路径
8  RKNN_MODEL = 'best.rknn'
9  IMG_PATH = './bus.jpg'
10 OUTPUT_PATH = 'result.jpg'
11
12 # 检测参数
13 OBJ_THRESH = 0.25
14 NMS_THRESH = 0.45
15 IMG_SIZE = 640
16
17 # 类别定义
18 CLASSES = ("person")
19
20 def sigmoid(x):
21     return 1 / (1 + np.exp(-x))
22
23 def xywh2xyxy(x):
24     # Convert [x, y, w, h] to [x1, y1, x2, y2]
25     y = np.copy(x)
26     y[:, 0] = x[:, 0] - x[:, 2] / 2 # top left x
27     y[:, 1] = x[:, 1] - x[:, 3] / 2 # top left y
28     y[:, 2] = x[:, 0] + x[:, 2] / 2 # bottom right x
29     y[:, 3] = x[:, 1] + x[:, 3] / 2 # bottom right y
30     return y
31
32 def process(input, mask, anchors):

```

图 3.21 部分检测代码

在将对应的模型文件还有图片文件地址都提供好以后，模型顺利地针对图片中的人员展开了检测，检测的结果如图 3.22 所示。在此次检测所呈现的结果当中，成功地把图片里的主要人员检测出来，这也就意味着模型在开发板上得以成功运行起来。



图 3.22 移植模型的测试结果

4 系统测试

在系统测试部分，主要是对模型的检测效果和离岗判定逻辑进行测试，主要分为本地测试与开发板测试两个部分，在本地测试阶段分为图片测试、视频检测以及实时监测三个部分，在开发板测试部分主要对移植模型的检测效果测试以及离岗行为判定逻辑的测试。

4.1 本地人员检测模型以及离岗检测算法测试

本地开展的测试主要针对项目前中期成果实施的测试，这种测试能够为模型在完成格式转换之后顺利在开发板上得以移植给予可靠的质量方面的保障，同时也能完成对模型性能方面的验证工作。在本地的环境当中对模型展开多维度的测试，具体涵盖功能测试、性能评估以及兼容性检验等内容。通过对人员检测模型进行本地测试能够提前将潜在的一些问题检测出来，进而针对这些问题做出优化与调整，最终保证模型在开发板上运行的时候所呈现出来的效果可以达到预期的那种效果。

在本地展开测试阶段的时候，重点围绕图片识别、视频识别以及摄像头识别这三个阶段来推进相关工作。在该阶段需要对模型的准确性予以严格的验证，保证它能够正确地处理输入的数据，并且输出的结果要符合预期的情况。与此同时，还要针对模型的运行效率展开综合的评估，这里面涉及到模型的检测效果以及推理速度等方面的指标，目的在于保障模型在开发板的硬件环境当中可以高效地运行起来。除此之外，还得细致地测试模型与开发板之间的适配性，要检测是不是会因为硬件方面存在差异或者软件环境有所不同，进而出现模型与开发板不兼容的状况。

4.1.1 图片检测

在图片检测环节当中仅对人员检测模型进行测试，我们需要去编写一段图片测试的代码，这段代码涵盖模型导入、待检测图片导入以及检测结果保存的功能。此阶段重点在于针对检测模型展开检测工作，所检测的类型是图片里的人，图 4.1 呈现的内容为图片测试的部分代码。

```
test.py x jiance.py x photo_test.py x shibie1.py x shibie.py x
1 import os
2 from ultralytics import YOLO
3
4 def main():
5     # 定义模型路径和图片路径
6     model_path = r"D:\Desktop\test1\test\runs\detect\train18\weights\best.pt"
7     image_path = r"D:\Desktop\bus.jpg" # 替换为你的图片路径
8
9     # 检查模型文件是否存在
10    if not os.path.exists(model_path):
11        print(f"模型文件未找到: {model_path}")
12        return
13
14    # 检查图片文件是否存在
15    if not os.path.exists(image_path):
16        print(f"图片文件未找到: {image_path}")
17        return
18
19    # 加载 YOLO 模型
20    try:
21        model = YOLO(model_path)
22    except Exception as e:
23        print(f"加载模型时出错: {e}")
24
25    if __name__ == "__main__":
```

图 4.1 图片测试部分代码

此次所采用的模型是 Pytorch 模型，在这种情况下，仅需简便地导入 YOLO 所依赖的库 ultralytics 对其予以声明就行。在完成待检测的图片的传入并且做好检测结果的相关设置之后，就能把检测结果存储在本地图，检测结果如图 4.2 所示。

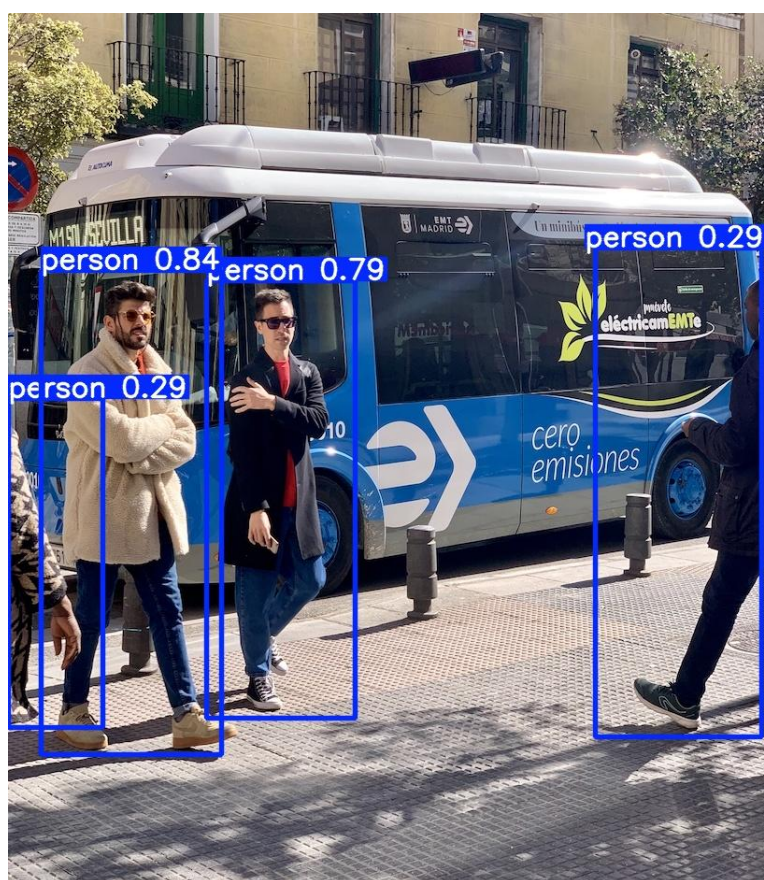


图 4.2 模型图片的检测效果

4.1.2 视频检测

在开展视频检测的时候，模型会针对视频里的每一个视频帧展开检测工作。在涉及修改的部分，需要将图片地址变更为准备进行检测的视频文件地址还需在此是加入离岗检测算法，修改之后的代码如图 4.3 所示。

```
def main():  
    # 定义模型路径和图片路径  
    model_path = r"D:\Desktop\test1\test\runs\detect\train18\weights\best.pt"  
    image_path = r"D:\Desktop\毕业设计\新建文件夹\人员在离岗识别\项目代码\data\videos\离岗识别.mp4"  
  
    # 检查模型文件是否存在  
    if not os.path.exists(model_path):  
        print(f"模型文件未找到: {model_path}")  
        return  
  
    # 检查图片文件是否存在  
    if not os.path.exists(image_path):  
        print(f"图片文件未找到: {image_path}")
```

图 4.3 视频检测部分代码

最终的视频检测结果如图 4.4、图 4.5 所示，其中图 4.4 为人员离岗时的检测结果，图 4.5 为人员在岗时的检测结果。从输出的情况来看，该模型针对视频里的人员所做出的检测结果以及离岗判定情况和预期的结果是相符的，成功地对人员的离岗行为进行检测并返回正确的人员在岗状态。

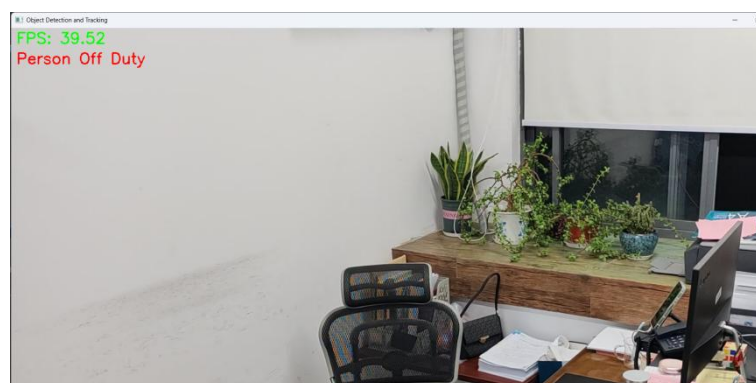


图 4.4 人员离岗检测结果

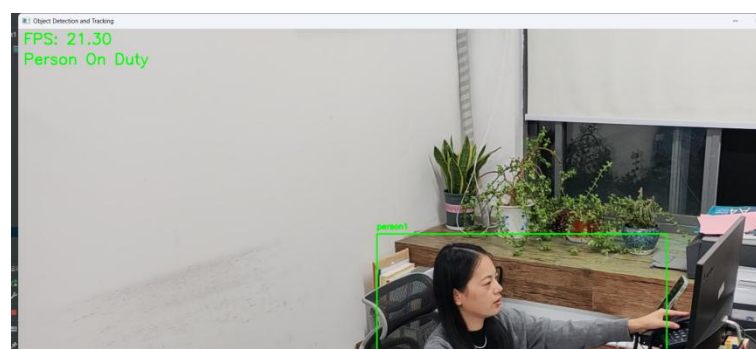


图 4.5 人员在岗检测结果

4.1.3 实时检测

实时检测属于项目本地测试的最后环节，它也体现着本地项目最终的运行成效。在这一环节当中，需要调用摄像头以便针对目标展开实时检测。该阶段的主要流程是先开启摄像头，接着对摄像头所抓取的画面予以读取，随后再针对获取到的画面内容实施检测。开启摄像头以及读取摄像头所捕捉画面的相关代码可参照图 4.5 所示。

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame")
        break

    # 将帧从 BGR 转换为 RGB (供 YOLO 使用)
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = model(frame_rgb)

    # 解码预测结果
    predictions = results[0].boxes.xyxy.cpu().numpy() # 转换为 numpy 数组
    confidences = results[0].boxes.conf.cpu().numpy()
    class_ids = results[0].boxes.cls.cpu().numpy()

    # 提取检测到的目标信息
    detections = []
    boxes = []
    scores = []
    classes = []
    for xyxy, conf, cls_id in zip(predictions, confidences, class_ids):
```

图 4.6 开启摄像头以及读取捕捉画面代码

实时检测的运行结果如图 4.7 所示，其中图 a 和图 b 分别为人员在岗时以及人员离岗时的检测结果。从结果图片可知，离岗判定逻辑在整个实时检测的这个过程中得以成功的实现，针对人员所做出的检测结果也和预期相符。

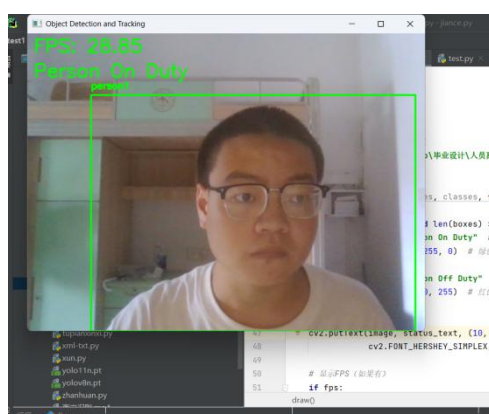


图 (a) 人员在岗识别结果

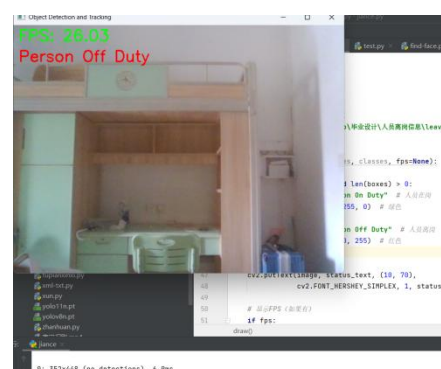


图 (b) 人员离岗识别结果

图 4.7 实时检测结果

4.2 开发板测试

在开发板测试阶段，主要的操作是在开发板里面编写一段测试代码，借助该代码最终呈现出来的检测结果去展示相关的演示成果，图 4.9 所示为图片测试的部分代码。

在这段代码当中，涵盖模型以及图片导入环节，还有测试框绘制方面的操作，另外也包含对检测结果加以处理等过程。在图片测试环节主要是对格式转换之后的检测模型进行测试，确保其依旧能对图片中的人员信息进行检测。

```

1 import os
2 import cv2
3 import time
4 import numpy as np
5 from rknnlite.api import RKNNLite
6
7 # 模型和图片路径
8 RKNN_MODEL = 'best.rknn'
9 IMG_PATH = './bus.jpg'
10 OUTPUT_PATH = 'result.jpg'
11
12 # 检测参数
13 OBJ_THRESH = 0.25
14 NMS_THRESH = 0.45
15 IMG_SIZE = 640
16
17 # 类别定义
18 CLASSES = ("person")
19
20 def sigmoid(x):
21     return 1 / (1 + np.exp(-x))
22
23 def xywh2xyxy(x):
24     # Convert [x, y, w, h] to [x1, y1, x2, y2]
25     y = np.copy(x)
26     y[:, 0] = x[:, 0] - x[:, 2] / 2 # top left x
27     y[:, 1] = x[:, 1] - x[:, 3] / 2 # top left y
28     y[:, 2] = x[:, 0] + x[:, 2] / 2 # bottom right x

```

图 4.9 图片检测部分代码

实际测试中，系统按照测试代码的设定，对导入的图片进行处理，处理结果如图 4.10 所示。最终的检测结果显示，主要的检测目标被清晰地识别和标注，测试框准确地勾勒出物体轮廓，检测结果的分类和置信度均在合理范围内，整体检测精度较高，基本满足开发板在测试阶段的功能验证需求。



图 4.10 图片测试结果

在转换模型的基础上将测试内容从图片改为视频并加入离岗判定算法，更改过后

的代码的绘制检测结果的部分代码如图 4.11 所示。在本部分，我们主要是绘制检测框以及人员在岗状态，通过对模型的检测结果进行检测来完成对人员离岗行为的检测。

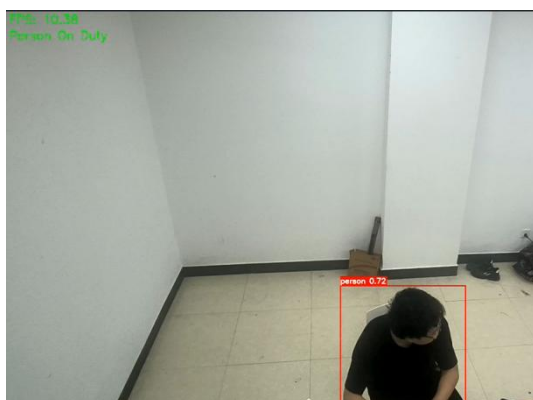
```

32 return boxes, classes, scores
33
34 def draw(image, boxes, scores, classes, fps=None):
35     """
36     绘制检测结果：边界框、类别标签、置信度、FPS
37     输出英文状态：Person On Duty (人员在岗)、Person Off Duty (人员离岗)
38     """
39     # 判定人员状态
40     if boxes is not None and len(boxes) > 0:
41         status_text = "Person On Duty" # 人员在岗
42         status_color = (0, 255, 0) # 绿色
43     else:
44         status_text = "Person Off Duty" # 人员离岗
45         status_color = (0, 0, 255) # 红色
46
47     # 显示人员状态
48     cv2.putText(image, status_text, (10, 70),
49                 cv2.FONT_HERSHEY_SIMPLEX, 1, status_color, 2)
50
51     # 显示FPS (如果有)
52     if fps:
53         cv2.putText(image, f"FPS: {fps:.2f}", (10, 30),
54                     cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
55
56     if boxes is None or len(boxes) == 0:
57         return image
58
59     # 定义类别颜色映射 (这里根据 CLASSES 索引, 当前只有 person 对应索引 0)
60     class_colors = {
61         0: (0, 0, 255), # 红色 (BGR格式) - person
62     }
63
64     # 绘制边界框和标签
65     for box, score, cl in zip(boxes, scores, classes):
66         top, left, right, bottom = box

```

图 4.11 检测结果绘制的代码

在我们传入待检测视频之后，模型成功的对待检测目标进行识别，检测结果如图 4.12 所示，其中图（a）和图（b）分别为人员在岗以及人员离岗检测结果。在此过程中，对人员离岗行为检测的逻辑被成功的实现并且被证实能够成功的对人员的在岗状态进行检测。



图(a) 人员在岗识别结果



图(b) 人员离岗识别结果

图 4.12 视频检测结果

5 结论与展望

5.1 总结

本次设计成功完成人员离岗行为检测系统的设计及实现工作。在构建检测系统的过程中，融合机器学习方面的知识、深度学习相关技术以及嵌入式技术等多种技术手段，并且采用 YOLO 目标检测算法以及 DeepSort 目标跟踪算法。在系统展开设计以及后续的实现进程当中，针对人员检测模型搭建以及离岗检测算法设计实现环节，着重对 YOLOv5 模型的具体结构还有与之相关的改进之处都给予说明。对于相关的数据集实施针对性的处理操作，将其标签文件转换成可用的 YOLO 格式，随后借助预训练模型来展开训练活动使得该模型针对人员目标所具备的识别能力有较为明显的提升，在这之后对于跟踪算法的接入以及离岗检测实现进行详细说明。在人员检测模型转换这个环节方面，先是把 Pytorch 模型依照相应顺序依次转换成为 ONNX 模型，接着再进一步转换为 RKNN 模型，成功使得模型能够在 RK3588 开发板上正常运行。在开发板移植部分完成开发板的环境配置工作，在该过程中完成对转换工具依赖库的安装以及开启摄像头驱动等工作，在此基础之上编写移植脚本，通过对搭建环境进行测试确保整个系统在实际的硬件环境当中能够保持稳定运行。

系统测试所得出的结果清晰地显示，不管是在本地测试环节涉及到的图片识别、视频识别以及实时识别方面，还是处于开发板测试环境之下，该系统都完全有能力准确地检测出人员的离岗行为。同时也为相关领域朝着智能化方向发展给予切实可行的技术方面的参照以及实践层面的经验借鉴，进而有力地推动人工智能技术在安防这一领域当中的实际应用。

5.2 展望

本次设计成功地完成了人员离岗行为检测系统的设计与实现，在实现系统的过程当中还是存在着一些局限性。在系统设计方面，系统一开始设计的时候，所针对的目标场景是对少数人的人员离岗行为展开检测。所以，要是遇到人数比较多的情况，又或者是存在较多重叠现象的时候，往往就会出现对部分目标检测出错的情况，甚至还会把其他的一些物体错误地识别成是人。再有就是，虽然引入 DeepSORT 算法提高了系统对于多目标的跟踪效果，可实际上，当前并没有与之匹配对应的多人离岗行为检

测的相关设置，依旧还是采用的单人离岗判定逻辑。最后在数据集处理这块，虽说处理过的数据集把大部分的有效参数都保留下来，但是因为转换代码以及转换逻辑方面存在着一些问题，所以对于部分图像边缘的标注框处理的效果并不是很好。在实际应用的时候，系统还得依据不同行业的离岗行为判定条件来进行差异化的设置。通过设定相应的离岗判定时长，这样才能准确地区分不同行业人员的离岗行为。往后的研究将会围绕着这些问题来开展，去探索更为先进的算法以及优化策略，从而进一步提升系统的鲁棒性与实用性，推动人员离岗行为检测技术朝着更高的水平去发展。

参考文献

- [1] 方东辉. 基于深度学习的工厂人员行为检测及管理系统[D]. 天津职业技术师范大学, 2024.
- [2] 王雪瑾. 基于深度学习的井下人员行为监测系统设计与实现[D]. 内蒙古大学, 2024.
- [3] 周益淇. 深度学习在计算机视觉领域的应用进展 [J]. 电子制作, 2018, (16): 30-31+29.
- [4] 马博洋. 基于计算机视觉的智能安防系统研究及应用. 吉林省, 大唐东北电力试验研究院有限公司, 2022-04-02.
- [5] 武熠明, 徐明智, 幸贞雄, 等. YOLOv4 算法在加油站卸油人员离岗检测中的应用研究 [J]. 工业安全与环保, 2023, 49 (06): 11-14.
- [6] 苏洪全, 谭蕾, 姜浩, 等. 基于视频的施工作业监护人离岗识别模型 [J]. 化工管理, 2024, (03): 81-86.
- [7] Saif H , Ghulam M , Asif R , et al. Multi-object tracking: a systematic literature review [J]. Multimedia Tools and Applications, 2023, 83 (14): 43439-43492.
- [8] Mahadevappa P , Amri A R , Alkaws G , et al. Analyzing Threats and Attacks in Edge Data Analytics within IoT Environments [J]. IoT, 2024, 5 (1): 123-154.
- [9] 刘丹. 嵌入式系统开发方法综述 [J]. 仪器仪表标准化与计量, 2008, (04): 8-11+17.
- [10] 高子召. RK3588 物体检测方法研究[D]. 长江大学, 2024.
- [11] 郑铁. 卷积神经网络在图像识别中的应用研究[C]// 天津市电子学会. 第三十八届中国（天津）2024' IT、网络、信息技术、电子、仪器仪表创新学术会议论文集. 德中(天津)技术发展股份有限公司; , 2024: 302-304.
- [12] 严家金. 基于卷积神经网络的图像分类算法研究[D]. 安徽理工大学, 2022.
- [13] 朱骏宇. 基于卷积神经网络的图像识别的技术分析[J]. 长江信息通信, 2023, 36(08): 66-68.
- [14] 谢红霞, 吴明晖. 从全连接网络到卷积神经网络的教学探讨 [J]. 福建电脑, 2020, 36 (07): 128-132.
- [15] 黄天才, 陈博, 张筱晨. 深度学习背景下的目标检测技术探究 [J]. 科技资讯, 2024, 22 (24): 41-43.
- [16] 任青阳, 王彦丁, 施俭. 卷积神经网络目标检测算法研究进展[J]. 科学技术与工程, 2024, 24(32): 13665-13677.
- [17] Fahad J , Alam J S , Naser A M , et al. Detection of mold on the food surface using YOLOv5 [J]. Current Research in Food Science, 2021, 4 724-728.
- [18] 胡瑞聪. 基于深度学习的多目标跟踪算法研究[D]. 广西民族大学, 2024.
- [19] 周翰祺. 基于无人机场景下的多目标跟踪技术研究[D]. 北京邮电大学, 2024.

致谢

当毕业设计临近完成之时，我满怀着感恩之情，想要把这份诚挚的致谢给予每一位曾给予我帮助的人。首要的是，我要诚挚感谢我的导师，在进行转换模型移植方面的研究以及撰写论文期间，导师凭借其深厚且渊博的专业知识、极为严谨的治学态度为我清晰地指明前行的方向，并且还耐心细致地解答我所遇到的每一个难题，由此给予我不断突破研究瓶颈的信心以及持续向前的动力。我还要感谢学院里的各位老师，他们在课堂之上悉心地教导我，从而为我筑牢坚实可靠的专业基础，课程中所积累起来的丰富知识也成为我能够顺利推进毕业设计的有力依托。与此同时，我要对同窗好友表达感谢之情，在我碰到困难之际，他们给予我暖心的鼓励与贴心的陪伴，当我的思路陷入阻塞的时候，他们又会和我展开热烈且富有成效的讨论，这使得我在这段科研的旅程当中不再感到孤单无助。最后，我尤其要感谢我的家人，正是他们在背后默默地给予支持以及毫无保留地付出，才让我可以心无旁骛、全身心地投入到研究工作当中。他们给予我的爱与充分的理解，无疑是我在不断前行道路上最为温暖且令人安心的港湾。毕业设计的完成并非是终点，而恰恰是全新征程的起始点，我将会带着这份深深的感恩之情，在未来的日子里继续坚持不懈地努力前行。